

华中科技大学

本科生毕业设计

采用元数据键值对存储的文件操作设计与实现

院 系 计算机科学与技术

专业班级 计科 2104

姓 名 岳云鹏

学 号 U202112345

指导教师 郭德纲

2025 年 13 月 32 日

学位论文原创性声明

本人郑重声明：所呈交的论文是本人在导师的指导下独立进行研究所取得的研究成果。除了文中特别加以标注引用的内容外，本论文不包括任何其他个人或集体已经发表或撰写的成果作品。本人完全意识到本声明的法律后果由本人承担。

作者签名： 2025 年 13 月 32 日

学位论文版权使用授权书

本学位论文作者完全了解学校有关保障、使用学位论文的规定，同意学校保留并向有关学位论文管理部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅。本人授权省级优秀学士论文评选机构将本学位论文的全部或部分内容编入有关数据进行检索，可以采用影印、缩印或扫描等复制手段保存和汇编本学位论文。

本学位论文属于 1、保 密 ☐，在 年解密后适用本授权书

2、不保密 ☒。

(请在以上相应方框内打“√”)

作者签名： 2025 年 13 月 32 日

导师签名： 2025 年 13 月 32 日

摘要

文件系统中元数据操作（创建、删除、重命名等）通常占到 I/O 负载的 60%–80%，其组织方式对系统整体吞吐有直接影响。传统文件系统使用 B+ 树管理目录项和 inode 索引，操作粒度为磁盘页，这使得即便只增删一条目录项，也需要把整页读出、修改后再写回。在元数据密集的场景下，这种就地修改会和目录级的锁竞争，写放大和并发受限同时出现。基于 LSM-tree（Log-Structured Merge-tree）的键值（Key-Value, KV）存储在这方面体现出优势：操作粒度从页降到单条记录，每条目录项保存为一个独立键值对，对目录项的增删只需一次追加写入；分散的小写入先汇入内存 MemTable，待其写满后顺序批量刷盘，把元数据常见的随机小写整理为顺序写入。

基于上述思路，本课题设计并实现了 RucksFS，这是一个以 RocksDB 为存储引擎、通过 FUSE 提供 POSIX 接口的用户态文件系统，覆盖 `create`、`mkdir`、`unlink`、`rmdir`、`rename`、`readdir`、`setattr`、`read`、`write` 等常见文件操作。实现过程中，本课题重点解决了三个核心问题：元数据如何建模为键值结构、并发访问下的语义正确性如何保证、共享父目录的并发竞争如何缓解。

存储建模上，系统按列族（Column Family, CF）划分数据：inode 属性和目录项各占一个独立列族，配以不同的布隆过滤器和压缩策略。其中目录项列族的键以“父目录 inode 号 + 子文件名”大端序拼接构成，每条目录项对应目录树中的一条父子边。`create`、`unlink` 各对应一次目录项的插入或删除，`rename` 则是一删一插，三者代价都是常数级。这种建模也使得同一父目录下的所有子项在键空间中相邻，目录遍历与单项查找就可以分别使用前缀扫描和点查完成。

并发安全方面，本课题使用 RocksDB 的悲观并发控制（Pessimistic Concurrency Control, PCC）事务：把 `create`、`rmdir`、`rename` 等操作中的“检查与修改”步骤封装在同一原子事务中，通过键级加锁消除多线程下的竞态。

事务保证了正确性，但 POSIX 语义要求 `create` 和 `unlink` 同步更新父目录的时间戳与链接计数，同一父目录在高并发下会成为写竞争热点。本课题为此引入增量更新与懒折叠机制（DeltaOp）：写入时不改动 inode 基值，只追加一条属性增量记录，将就地改写转为追加写入；读取时把基值与未折叠的增量实时

叠加后返回，保证调用方看到的是最新状态；后台线程定期扫描累积的增量，超过阈值时一次折叠回基值。该机制在路径上消除了父目录的写锁争用。

正确性测试方面，在分布式部署下运行 `pjdfstest` 套件共 398 个用例；排除依赖 `mknod`、`mkfifo` 等本文不支持接口的用例后，剩余 182 个用例全部通过，通过率 100%。

性能方面，使用 `mdtest` 在 1 台 64 核服务器与最多 96 台 2 核客户端构成的集群上压测元数据操作，并以 NFS、JuiceFS 作为对比基线。相对基于 `ext4` 的 NFS v4.2，RucksFS 在低并发下略慢，并发上升到 32 个客户端时 `create` 反超约 4.6 倍，印证了 KV + LSM 路线在高并发元数据负载下相较传统方案的优势；相对同为 KV 元数据方案的 JuiceFS+TiKV，RucksFS 在 `create`、`remove`、`stat` 上分别稳定领先约 1.5–1.7、1.8–2.2 和 1.2 倍，说明本课题在 KV 路线内部也做到了不错的实现水平。

此外，为了在更高并发下考察 DeltaOp 的实际作用，本文编译了两个 RucksFS 版本，二者仅在是否启用 DeltaOp 上有差别，作为消融对照。两个版本在每个进程使用独立父目录、彼此不产生冲突时表现基本一致，而一旦落到共享父目录场景，启用 DeltaOp 的版本表现出明显更好的可扩展性：并发数超过 96 后，不启用版本因反复的事务冲突无法继续增长，稳态吞吐保持在 12 000 ops/s 附近；启用版本则继续扩展至 38 000 ops/s，在 384 并发下 `create` 与 `remove` 吞吐分别达到前者的 2.96 倍和 2.63 倍。这证明了 DeltaOp 在高并发场景下的实际价值。

关键词： 文件系统；元数据管理；键值存储；LSM-tree；增量更新；悲观并发控制；FUSE

Abstract

Metadata operations such as file creation, deletion, and renaming typically account for 60%–80% of file system I/O, so the way metadata is organised directly determines overall throughput. Traditional file systems manage directory entries and inode indexes with B+ trees at page granularity: even a single-entry change forces a page read, modification, and write-back. Under metadata-intensive workloads, this in-place update also stacks on top of directory-level lock contention, so write amplification and concurrency bottlenecks arise at the same time. A Key-Value (KV) store backed by a Log-Structured Merge-tree (LSM-tree) is a better fit on both counts: the operational granularity drops from a page to a single record, each directory entry is stored as an independent key-value pair, and insertions or deletions of a directory entry only require a single append; scattered small writes are first buffered in an in-memory MemTable and later flushed to disk in sequential batches, turning the random small writes common in metadata workloads into sequential ones.

Building on this idea, this work designs and implements RucksFS, a user-space file system backed by RocksDB and exposed as a POSIX mount via FUSE, covering the common file operations `create`, `mkdir`, `unlink`, `rmdir`, `rename`, `readdir`, `setattr`, `read` and `write`. During the implementation, this work addresses three core problems: how to model metadata as key-value structures, how to guarantee semantic correctness under concurrent access, and how to relieve concurrent contention on shared parent directories.

For storage modelling, the system partitions data across Column Families (CFs): inode attributes and directory entries occupy separate CFs, each tuned with its own Bloom filter and compaction policy. Keys in the directory-entry CF are formed by concatenating the parent inode number and the child filename in big-endian order, so every directory entry corresponds to an edge in the directory tree. `create` and `unlink` each correspond to a single directory-entry insertion or deletion, while `rename` is one deletion plus one insertion; all three carry constant cost. All child entries of the same

parent are adjacent in key space, so directory traversal and single-entry lookup are served by prefix scan and point query, respectively.

For concurrency safety, this work uses RocksDB Pessimistic Concurrency Control (PCC) transactions: the check-then-modify sequence of `create`, `rmdir`, and `rename` is wrapped inside a single atomic transaction, and per-key locking removes the races between the check and the mutation.

Transactions take care of correctness, but POSIX still requires `create` and `unlink` to synchronously update the parent’s timestamps and link count, which turns the parent inode into a write hot spot under concurrency. This work addresses the hot spot with a delta update and lazy folding mechanism (DeltaOp). Each write appends a small delta record without touching the base inode value, turning in-place rewrites into appends; each read merges the base value with any outstanding deltas on the fly, so callers always observe up-to-date attributes; a background thread periodically folds accumulated deltas back into the base value once a threshold is reached. Along the critical path, this eliminates write-lock contention on the parent directory.

For correctness, a full `pdfstest` run in a distributed deployment executes all 398 cases; excluding those that depend on `mknod`, `mkfifo` and other interfaces this work deliberately leaves out, the remaining 182 cases all pass, giving a 100% pass rate within scope.

For performance, `mdtest` is run on a cluster of one 64-core server and up to 96 two-core clients, with NFS and JuiceFS as baselines. Against NFS v4.2 on ext4, RucksFS is slightly slower at low concurrency but overtakes NFS on `create` by roughly $4.6\times$ once the client count reaches 32, confirming the advantage of the KV + LSM approach over the traditional scheme under concurrent metadata workloads. Against JuiceFS+TiKV, a fellow KV-based metadata design, RucksFS leads by a stable $1.5\text{--}1.7\times$, $1.8\text{--}2.2\times$, and $1.2\times$ on `create`, `remove`, and `stat` respectively, showing that this work is also competitive within the KV route itself.

To examine DeltaOp’s effect under higher concurrency, two RucksFS builds are compiled as an ablation; the two differ only in whether DeltaOp is enabled. When

each process operates under its own parent directory and the two builds do not actually contend, they perform almost identically; once a shared parent directory is involved, the DeltaOp-enabled build shows markedly better scalability: beyond a concurrency level of 96, the disabled build is bounded by repeated transaction conflicts and its throughput stabilises around 12,000 ops/s, whereas the enabled build continues to scale to 38,000 ops/s; at a concurrency of 384, the create and remove throughputs of the enabled build reach $2.96\times$ and $2.63\times$ those of the disabled build respectively. This confirms the practical value of DeltaOp under highly concurrent workloads.

Keywords: File System, Metadata Management, Key-Value Storage, LSM-tree, Delta Update, Pessimistic Concurrency Control, FUSE

目 录

摘 要	I
Abstract	III
1 绪 论	1
1.1 课题背景、目的与意义	1
1.2 国内外研究现状	6
1.3 论文的主要内容与结构	14
2 相关技术基础	17
2.1 FUSE 用户态文件系统框架	17
2.2 RocksDB 存储引擎	21
2.3 POSIX 文件系统语义	24
2.4 本章小结	26
3 元数据组织与事务模型	27
3.1 设计目标	27
3.2 元数据存储模型	27
3.3 DeltaOp 增量更新与懒折叠	31
3.4 悲观事务与并发正确性	33
3.5 核心操作实现	35
3.6 本章小结	38
4 系统实现	39
4.1 实现概览	39
4.2 客户端：从 FUSE 到两个服务	40
4.3 MetadataServer 实现	43
4.4 DataServer 实现	44
4.5 RPC 与连接层	45
4.6 错误处理与恢复	47
4.7 本章小结	48

5	测试与结果分析	50
5.1	研究目标与评估线索	50
5.2	POSIX 合规性评估	50
5.3	元数据性能评估	54
5.4	本章小结	68
6	总结与展望	70
6.1	工作总结	70
6.2	不足与展望	71
	致谢	73
	参考文献	75

1 绪 论

大规模存储系统中, `stat`、`open`、`readdir` 等元数据操作占到文件系统 I/O 总数的 60%–80%^[1], 元数据路径的组织方式在很大程度上决定了系统整体吞吐。传统文件系统用 B+ 树及其变体维护目录项和 `inode` 索引, 操作粒度是磁盘页, 即使只改动一条目录项也要读出整页、修改后再写回; 在高并发小文件场景下, 这种就地修改还会与目录级锁竞争叠加, 写放大与并发受限两个问题同时出现。基于 LSM-tree 的键值 (KV) 存储把操作粒度从页降到单条记录, 并以顺序写入代替随机写入, 为该问题提供了另一条可行路径。

1.1 课题背景、目的与意义

1.1.1 研究背景

大规模模型训练、容器化微服务和云端数据湖等场景有一个共同特征: 它们产生并消费海量小文件。一次分布式训练作业可能在 `checkpoint` 阶段写出数十亿个参数分片; Kubernetes 集群的 Pod 在启动过程中需要创建和读取大量配置文件与临时卷。百度沧海存储团队在 Mantle^[2] 中给出了一组具体的数据: 其云对象存储管理的文件对象数已达万亿量级, 元数据请求规模每年仍有数倍增长。在此类海量小文件负载下, 系统性能瓶颈通常不在数据传输带宽, 而在元数据路径上。Ren 和 Gibson^[1] 的测量表明, `stat`、`open`、`readdir` 等元数据操作占文件系统全部 I/O 操作的 60%–80%。单个热点目录下的文件数一旦达到百万级, 元数据查询与更新的延迟会直接压低系统吞吐上限。

现有的本地文件系统 (`ext4`、`XFS`) 采用 B+ 树或其变体 (如 `ext4` 的 H-tree) 组织目录项和 `inode` 索引。以最简单的 `create("/mnt/dir/newfile")` 为例, 这一操作在 `ext4` 上涉及 5 类磁盘区域、至少 6 次写入:

- (1) **Journal 事务起始**。`ext4` 在 `ordered` 模式下首先向日志区域写入一条事务描述符块 (descriptor block), 标记本次操作的开始。日志区域位于分区上的固定偏移量, 与用户数据不相邻。
- (2) **inode bitmap**。从新文件所属的块组中查找空闲 `inode` 位: 读取 `inode` 位图块 (4 KiB), 翻转对应比特位, 写回。位图块的磁盘位置取决于块组编号,

通常与上一步的日志区域相距甚远。

- (3) **inode 表**。在 inode 表中初始化新分配的 inode 条目：写入 mode、uid、gid、时间戳等共 256 字节的属性数据。inode 表位于该块组的固定偏移处，又是一次不同位置的写入。
- (4) **目录数据块**。在父目录 /mnt/dir 的数据块中插入一条目录项 (dirent)。ext4 的 H-tree 是 B+ 树的哈希变体，插入时可能需要读取并修改叶子节点的数据块，若叶节点已满还要分裂出新块。目录数据块散布在父目录所属的块组中。
- (5) **父目录 inode**。更新父目录的 mtime (修改时间) 和 ctime (状态变更时间)，如果创建的是子目录还要递增 nlink (链接计数)。父目录 inode 与新文件 inode 可能不在同一个块组中。

最后，ext4 把上述所有修改的元数据块写入 journal 区域，追加一条 commit 块，调用 fsync 确保日志落盘，然后再将实际数据写回原位 (称为 checkpoint)。单次文件创建共产生至少 6 次磁盘写入，分布在 inode bitmap、inode 表、目录数据块、父目录 inode 和 journal 等互不相邻的磁盘区域上，每一次都是随机 I/O。图 1-1 展示了这一过程。

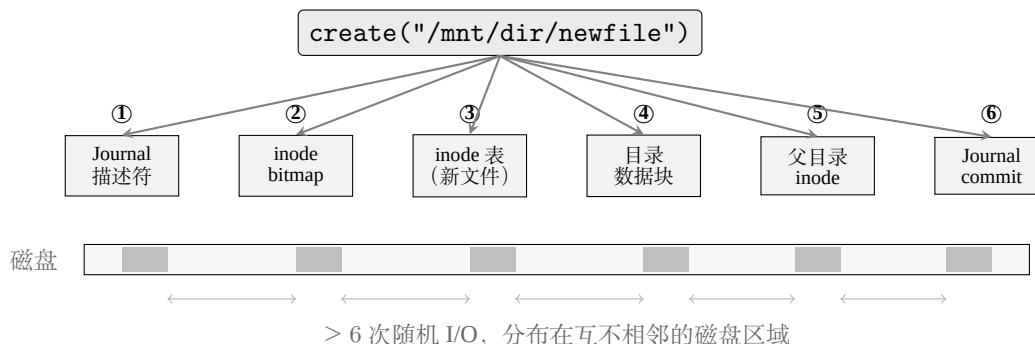


图 1-1 ext4 中一次 create 操作的磁盘写入路径。单次文件创建至少触及 6 个分散的磁盘区域，每次写入都是随机 I/O。

这一路径上的主要开销来自粒度不匹配。一次文件创建真正需要写入的有效数据不超过 300 字节 (256 字节的 inode 加几十字节的目录项)，但 ext4 的操作粒度是 4 KiB 的磁盘页：即使只修改 256 字节的 inode，也需要读出整个 4 KiB 页、修改后再写回。Journal 模式下该页还会被写两次 (一次进 journal，一次落原位)。如果目录的 H-tree 恰好触发节点分裂，写入量还会再翻倍。单次操作的

开销尚可接受，但当每秒需要执行数万次 `create` 时，写放大的累积效应会明显拖累系统吞吐。

并发场景下情况更为严峻：`ext4` 对目录数据块的修改受目录级互斥锁 (`i_rwsem`) 保护，同一目录下的所有文件创建操作必须串行通过这把锁。即便底层存储是 NVMe SSD，能提供数十万 IOPS，单目录下的创建吞吐仍会被该锁压到远低于硬件能力的水平。

基于 LSM-tree (Log-Structured Merge-tree)^[3] 的键值 (KV) 存储为该问题提供了一条不同的技术路线。KV 存储把操作粒度从 4 KiB 的磁盘页细化到单条记录：一个目录项即一个键值对，写入时只做一次内存追加，不涉及 bitmap 翻转、journal 双写或 4 KiB 对齐的页填充。MemTable 写满之后一次性顺序刷盘，将原本分散的随机写归并为连续的顺序 I/O。TableFS^[1] 在 2013 年把全部元数据存入 LevelDB^[4]，通过 FUSE 挂载，小文件创建吞吐达到同期 `ext4` 的 2-3 倍，首次验证了这条路线的可行性。此后 RocksDB^[5] 在 LevelDB 基础上加入了列族、事务和前缀布隆过滤器等特性；Apache Ozone^[6] 和 Facebook Tectonic^[7] 先后在 EB 级生产环境中使用 RocksDB 作为元数据引擎，验证了 KV 元数据方案在工业规模下的整体可行性。不过两者都不对外提供完整的 POSIX 接口：Ozone 面向 S3 与 Hadoop API，Tectonic 面向 Facebook 内部的对象存储协议，键编码、父目录写放大、跨键事务等在 POSIX 语义下才集中出现的问题在这条路径上并未完整展开。

1.1.2 面临的问题和挑战

将 KV 引擎用于文件系统元数据管理，需要解决三个核心设计问题 (图 1-2)。

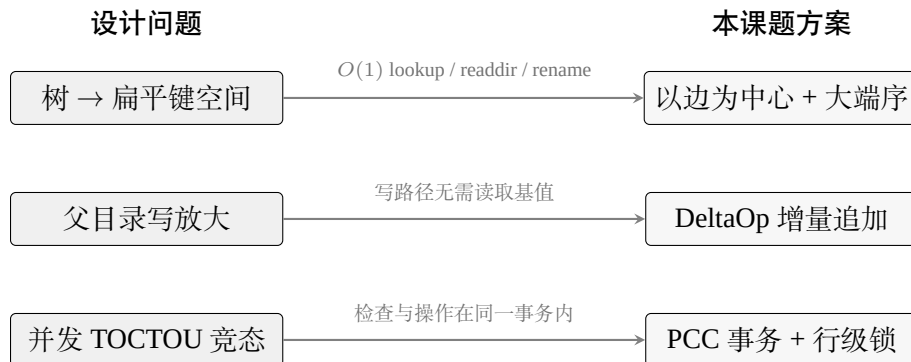


图 1-2 本课题面临的三个设计问题及对应解决方案。

问题一：树形目录到扁平键空间的映射。KV 引擎看到的是字节串到字节串

的扁平映射，而文件系统对外暴露的命名空间是一棵树。如何为树上每个对象设计键，使得创建、查询、遍历、重命名都能高效完成，是首先需要想清楚的问题。

以 `create` 为例：`ext4` 中一次文件创建要写 6 个分散的磁盘区域；在 KV 存储中，如果把每条目建模为目录树上的一条父子边、用一个键值对表示，`create` 就是插入一条记录，一次内存追加即可。但这一简化能否成立，取决于键的编码方式。键编码需要同时满足三项要求：其一，`create` 和 `unlink` 为 $O(1)$ 的单条增删；其二，`readdir` 要让同一父目录下的子项在键空间中相邻，使得一次前缀扫描即可完成；其三，`lookup` 在拿到父 `inode` 号和文件名后能通过一次点查定位。若直接以全路径 (`/a/b/c`) 作为键，`create` 尚可，但一次 `rename` 需要递归改写整棵子树的所有键，开销随子树规模线性增长。`TableFS`^[1] 以「父 `inode` + 文件名」拼接作为键（这一形式本质上即把目录项视为目录树上的一条父子边），`create`、`lookup`、`rename` 均为 $O(1)$ ；但其 `inode` 号采用原生字节序存储，同一目录的条目在 `LSM-tree` 中未必物理相邻，前缀布隆过滤器也就无法发挥作用。因此键编码还需保持字节序与数值序一致 (`byte-order preserving`)，否则范围查询会给出错误结果。

问题二：父目录更新引发的写放大。 `POSIX` 语义^[8] 要求目录项发生变化时同步更新父目录的 `mtime` 和 `ctime`；`mkdir` 和 `rmdir` 这类子目录增删还会同步改动父目录的 `nlink`。若沿用传统的 `inode` 原地更新思路建模，每一次 `create`、`unlink`、`mkdir`、`rmdir` 都要先读出父目录 `inode` 值、修改其中几个字段，再写回同一条键。对热点目录 (`/tmp`、CI 产物目录、模型训练的 `checkpoint` 目录) 而言，这条键会承受大量连续的读-改-写，相邻两次操作之间还可能被别的 `create` 插入，在事务并发下导致冲突重试。在 `RocksDB` 中，对同一个 `key` 的密集 `Get+Put` 会在同一条 `key` 上积累大量历史版本，`compaction` 需要反复把这些版本归并为最新值，同时点查时读放大也随之上升。KV 层面的写路径本身仍是顺序追加，但这种“写-改-写同一条 `key`”的模式把原本可以均匀分散到整个键空间的顺序追加收窄到了一条键上，热点目录属性的更新也就失去了 KV 建模本应带来的那部分优势。

问题三：并发目录操作的正确性。 多个线程同时操作同一目录时，「检查

条件」和「执行修改」之间存在一段时间窗口，即 TOCTOU (Time-of-Check-to-Time-of-Use) 竞态。例如线程 A 执行 `rename` 时已确认目标路径不存在，但在其写入新目录项之前，线程 B 已在同一位置创建了同名文件：KV 层的写入本身是 last-write-wins 的，两条操作谁后到谁生效，但上层看到的语义已经不自洽：A 认为自己完成了一次无冲突的 `rename`，B 却又让同名文件重新出现在该位置。内核文件系统由 VFS 层的 inode 互斥锁将目录操作串行化；KV 引擎之上没有这一层保护，只能依赖存储层自身的事务机制。锁粒度的选择（锁整个目录还是仅锁涉及的 key）又直接关系到并发度与实现复杂度之间的取舍。

1.1.3 课题目的与意义

本课题的目标是：设计并实现一个基于 RocksDB 的用户态 FUSE 文件系统 RucksFS，针对上述三个问题给出可落地的方案，并通过实验验证方案是否达到预期。

技术选型。 元数据引擎选用 RocksDB，理由是其特性与上述三个问题对应较好：Column Family 可以把 inode 属性、目录项、增量记录放在相互独立的键空间中各自配置参数；前缀迭代器和前缀 Bloom 过滤器能够支撑目录前缀扫描；悲观事务 (TransactionDB) 提供了行级锁与冲突检测，可用于避免 TOCTOU 竞态。对外接口选择 FUSE 出于工程层面的考虑：用户态开发迭代快、调试方便，与 JuiceFS^[9]、CubeFS^[10] 的接入方式一致，便于在相同工具链下进行性能对比。实现语言采用 Rust^[11]：一方面文件系统这类基础设施对内存安全要求较高，Rust 在编译期即可排除大部分相关缺陷；另一方面其 `async/await` 生态 (tokio) 与 FUSE 的异步处理模式契合度较高。

解决思路。 针对上述三个问题，本课题分别给出三项设计。键编码采用「以边为中心」的思路，将父 inode 号与子文件名按大端序拼接作为目录项键，使文件操作对应目录树中一条边的常数时间增删。父目录写放大的问题通过增量更新 (DeltaOp) 解决：写入时只追加一条变化量，不修改基础 inode 值，由后台线程定期将累积的增量折叠回基值。并发正确性由 RocksDB 悲观事务保证，将检查与修改封装在同一事务内；按 inode ID 排序加锁以预防死锁。三项设计的细节在第 3 章展开。

课题意义。已有 KV 元数据研究中, TableFS 阶段的系统普遍缺少事务能力; IndexFS 把重心放在跨节点扩展上; Mantle 在 EB 级规模上给出了完整的工业方案, 但依赖自研分布式事务数据库 TafDB 与一套较复杂的缓存一致性协议。RocksDB 自 6.0 版起提供了成熟的悲观事务接口, 把它系统性地应用到 POSIX 文件操作元数据层、并配上增量更新与合适的键编码的公开工作目前尚少。本课题即沿这条路线给出一个可复现的原型: Rust 实现、三段式分布式部署形态、完整的 POSIX 操作集合和配套实验; 后续加入元数据分片、副本容错或元数据预取等工作, 可直接基于这套骨架展开。

预期交付物。其一, 按 FUSE 客户端、MetadataServer 和 DataServer 解耦的 RucksFS 原型, 支持 gRPC 远程通信与分离部署; 其二, pjdftest POSIX 合规性测试结果; 其三, 在同一硬件条件下与 JuiceFS+TiKV、NFS v4.2 的元数据吞吐对比数据, 以及 Delta 与 NoDelta 两种构建变体的内部对照数据。

1.2 国内外研究现状

用 KV 引擎管理文件系统元数据的研究始于 2013 年, 至今已积累了从单机原型到 EB 级生产系统的丰富实践。

1.2.1 基于 KV 存储的元数据管理研究

TableFS^[1] 是这一方向最早的系统性工作。2013 年, Carnegie Mellon University 的 Ren 和 Gibson 把 ext4 的目录项和 inode 信息整体存入 LevelDB, 并通过 FUSE 挂载为标准文件系统。TableFS 的设计思路直接: 以 (父 inode 号, 文件名) 的拼接作为 LevelDB 的键, 以序列化后的 struct stat 加上可选的内联数据作为值。在小文件 (< 4 KB) 场景下, TableFS 的 create 吞吐量是同期 ext4 的 2-3 倍, 首次验证了 KV 存储用于元数据管理的可行性。但 TableFS 也暴露出若干缺陷: 其一, readdir 操作需要扫描 LevelDB 的全部键空间并过滤出指定父目录的条目, 当文件系统中的文件总数增长到百万级别时, readdir 的延迟会明显上升; 其二, LevelDB 本身不提供事务语义, 多线程写入的正确性完全依赖外部加锁; 其三, 键编码中父目录 inode 号使用原生字节序存储, 导致同一目录下的条目在 LSM-tree 中不一定连续排列, 无法利用前缀迭代器或前缀布隆过滤器加速查找。另外, TableFS 也没有处理父目录 inode 更新带来的写放大问题。

同一研究团队在次年推出了 IndexFS^[12]，把 TableFS 的思路扩展到分布式环境。IndexFS 在每个元数据节点上运行一个 LevelDB 实例，目录项通过 GIGA+ 动态哈希分区算法分布到不同节点。其关键贡献是引入了列式目录布局（columnar directory layout）：将 `readdir` 所需的信息（文件名列表）与 `getattr` 所需的信息（inode 属性）分开存储在不同的 SSTable 列中，使得目录遍历只需读取文件名列表，跳过体积更大的 inode 属性数据。IndexFS 在 128 节点集群上实现了每秒超过 100 万次 `create` 操作，获得了 SC'14 最佳论文奖。不过 IndexFS 的部署要求较高：每个节点需要独立维护一个 LevelDB 实例和一套目录缓存状态机，还需要一个独立的 GIGA+ 组件处理跨节点目录分裂，对中小规模场景来说系统复杂度偏重。

LocoFS^[13] 在 2017 年观察到文件访问的时间局部性和空间局部性，设计了一种松耦合的元数据分布策略，按哈希将热点目录的元数据分配到固定服务器。Xu 等人^[14] 在 2014 年提出了面向 EB 级规模的子树粒度元数据划分方案。这些工作侧重多节点元数据划分与扩展策略，与本课题的关注点（在分布式访问路径下如何围绕 RocksDB 组织 POSIX 元数据操作）不在同一层面。

百度沧海·存储团队在 SOSP'25 上发表的 Mantle^[2] 是这一路线的最新进展。Mantle 为云对象存储提供层级命名空间支持，底层持久化引擎 TafDB 采用三个列族组织元数据：AccessCF 以（父 inode 号 + 文件名）为键存储目录项，AttrCF 以 inode 号为键存储属性，DeltaCF 以（inode 号 + 时间戳）为键追加增量更新记录。DeltaCF 的 Delta Record 机制与本课题的设计动机一致：当多个客户端并发在同一目录下创建文件时，父目录的子项计数和 `mtime` 不再以读-改-写方式更新，而是各自追加一条增量记录，由后台线程异步合并到 AttrCF 中的基准值，从而消除热点目录属性上的写竞争。

Mantle 还在 TafDB 之上叠加了 IndexNode 内存缓存层，用于加速长路径解析和权限检查，在百度内部 EB 级数据湖上实现了相比旧方案 115 倍的吞吐量提升。不过 Mantle 依赖的 TafDB 是百度自研的类 Spanner 分布式事务数据库，IndexNode 与 TafDB 之间的缓存一致性协议也较复杂，整套系统并不开源。本课题借鉴 Mantle 的增量更新思路，但把它落到 MetadataServer 本地的 RocksDB 事务上实现：元数据仍由单个 MetadataServer 统一维护，DeltaOp 追加、懒折叠和

折叠结果缓存全部在同一 RocksDB 实例内完成，不再依赖外部分布式事务数据库和跨服务的缓存一致性协议，以较低的系统复杂度复现其核心收益。

1.2.2 云原生文件系统中的元数据方案

在 KV 元数据研究逐步成熟的同时，工业界的分布式文件系统也在快速演进。本课题在性能实验中选择 JuiceFS 作为实验主对比对象，NFS v4.2 作为传统本地文件系统代表，这里简要介绍它们与其他几个代表性系统的元数据方案，为后续对比提供背景。

早期的分布式文件系统以集中式元数据管理为主：GFS^[15] 和 HDFS^[16] 把全部元数据驻留在单节点内存中，Ceph^[17] 用动态子树划分将目录树分配到多个元数据服务器。这些系统在 PB 级规模上运转良好，但元数据的可扩展性始终受限于内存容量或子树迁移开销。分布式 KV 存储（以 Amazon Dynamo^[18] 的可用性优先设计为代表）逐步成为上层文件系统的新型后端选择之后，Apache Ozone^[6] 和 Facebook Tectonic^[7] 在 2020 年前后将元数据引擎替换为 RocksDB，分别管理了 EB 级数据；不过两者都不提供完整 POSIX 接口，其中 Ozone 面向 S3 和 Hadoop API，Tectonic 面向 Facebook 内部的对象存储协议。

JuiceFS^[9] 是国内使用较广的开源 POSIX 文件系统，其架构将元数据和数据完全分离：元数据存储于 Redis、TiKV 或关系型数据库（MySQL、PostgreSQL 等）之中，数据块写入 S3 兼容的对象存储。这种解耦方式允许用户按自身条件选择后端组合；代价是性能和可用性完全取决于外部组件：Redis 的单线程模型在元数据操作超过每秒数万次时会出现瓶颈，TiKV 则引入 Raft 共识协议带来的额外延迟。JuiceFS 官方基准显示，Redis 后端在单客户端下的文件创建吞吐约为 1,410 ops/s，MySQL 后端仅约 350 ops/s。JuiceFS 的 FUSE 客户端使用 Go 实现，Go 运行时的垃圾回收在极端负载下存在引入延迟抖动的可能。

CubeFS^[10]（最初名为 CFS）于 2019 年在 SIGMOD 发表，后捐赠给 CNCF 并于 2024 年成为毕业项目。与 JuiceFS 依赖外部数据库不同，CubeFS 自带元数据引擎：每个元数据分区（MetaPartition）使用内存中的 B-tree 维护目录结构，多副本之间通过 Multi-Raft 协议保持一致性。CubeFS 的读路径延迟在微秒级，但写路径受 Raft 日志同步所限，典型延迟在毫秒级；内存 B-tree 也限制了单节点

可管理的元数据量，按每个 inode 约 300 字节内存估算，十亿文件需要约 280 GB 内存。

上述系统按元数据承载方式可归为三类：内存驻留型 (GFS、HDFS、CubeFS)、外部数据库型 (JuiceFS)、嵌入式 KV 型 (Ozone、Tectonic)。三类方案侧重点不同，但都围绕分布式环境组织元数据服务：要么把一致性和事务交给外部数据库，要么把复制、分片和故障恢复做进文件系统自身。由此引出一个更具体的问题：在已经采用分布式部署的前提下，元数据引擎本身该如何围绕文件操作做定制，而不是完全受制于通用数据库或共识框架的语义与开销。

1.2.3 元数据引擎的深度优化研究

与分片扩展路线并行的另一条研究路线更关注元数据引擎本身的执行效率，即先缩短单次操作的关键路径，再讨论系统如何扩展。SingularFS^[19] 是这一路线的代表。

SingularFS 在 ATC'23 上展示了一个激进的设计选择：仅用单个元数据服务器支撑十亿级文件规模的分布式文件系统。其核心优化分为三项：其一，无日志元数据操作 (log-free metadata operations)，通过精心设计的持久化顺序消除崩溃一致性日志的开销；其二，层次化并发控制，对目录树的不同层级使用不同粒度的锁，避免粗粒度锁的串行化瓶颈；其三，NUMA 感知的 inode 分区，将 inode 地址空间按 NUMA 节点划分，减少跨节点内存访问。在配备 RDMA 网络和持久内存的硬件环境下，SingularFS 的单节点元数据吞吐量远超传统多节点方案。这一结果对本课题的参考不在于「把系统做成单机」，而在于即便整体以分布式方式部署，元数据服务内部仍值得围绕关键路径做深度优化。RucksFS 即沿此思路，在 MetadataServer 的 RocksDB 事务、增量更新和键编码上做针对性设计，而不是把全部语义都外包给通用分布式 KV。

Wang 等人^[20] 在 EuroSys'23 上发表的 CFS 对元数据服务中临界区 (critical section) 的范围做了精细化分析。传统方案处理 rename 等涉及多个目录的操作时多采用粗粒度的目录级锁，导致不相关的操作被串行化。CFS 用形式化方法识别出各个 POSIX 操作中真正需要互斥的 key 集合，将临界区缩减到最小，rename 密集型工作负载的吞吐提升了 3.2 $\times$ 。本课题的 PCC 事务设计借鉴这一思路，只

对实际涉及的 key 加排他锁，而非锁定整个目录。

Yang 等人^[21]在 2020 年针对大规模目录中逐个执行 stat、open 效率低下的问题，设计了将 readdir+stat 复合操作在内核内合并执行的批量接口，减少用户态/内核态切换次数。这一批量化思路直接影响了 RucksFS 的 readdirplus 实现，即在一次 FUSE 调用中同时返回目录项和完整的 inode 属性，避免后续逐个 getattr 的往返。

操作系统层面也有相关工作。FetchBPF^[22]发表于 ATC'24，利用 eBPF 在 Linux 内核中实现可定制的文件访问预取策略，在顺序读负载上接近理论带宽。FBMM^[23]展示了将 Linux VFS 接口用于非传统用途的可能性，即把内存管理器实现为文件系统，利用 VFS 的可扩展性在不修改内核核心代码的前提下引入新策略。这两项工作与本课题不直接重合，但都为 RucksFS 的后续优化提供了方向：例如在 FUSE 的 /dev/fuse 通道上通过 eBPF 预取即将被访问的元数据，有可能降低用户态守护进程的响应延迟。

MetaHive^[24]在 2024 年针对 KV 存储引擎在元数据场景下的缓存效率问题展开研究。通用 KV 存储的缓存替换策略（如 LRU）未考虑文件系统的访问局部性：同一目录下的 inode 被连续访问的概率远高于随机 key。MetaHive 据此设计了元数据感知的缓存分区方案。本课题与之方向相近但关注点不同：RucksFS 在应用层维护的 InodeFoldedCache 侧重缓存折叠后的 inode 值，避免读路径反复扫描 delta_entries 并重复折叠，属于针对 DeltaOp 机制的特化缓存，而非按目录局部性分区。沿「为文件系统元数据定制执行路径」这一方向，FUSEE^[25]将 KV 元数据服务下沉到内存解耦架构上；Dong 等人^[26]在 ICCD 2023 上讨论了完整路径索引下的低延迟元数据服务设计；SwitchFS^[27]则在交换机侧协调元数据异步更新。这些工作的方向与本课题并不完全重合，但都表明元数据引擎本身仍有较大的优化空间。

在国内，龚拓宇^[28]在 2020 年的清华大学硕士论文中结合 RDMA 技术与 RocksDB 设计了分布式元数据服务原型 RocksMeta，在单节点上实现了超过 10 万 IOPS 的元数据操作吞吐量；其键编码方案与本课题有相似之处，但侧重网络传输优化（RDMA 旁路内核协议栈），本课题侧重存储层的写放大优化。范立衡等人^[29]较早探索了 KV 存储用于元数据集副本一致性的管理策略。吴雨飞等

人^[30] 在 2025 年针对基于 TiKV 的分布式文件系统元数据缓存一致性问题，提出了并发广播与惰性删除相结合的轻量级维护机制。

表 1-1 按「对本课题架构选择的直接影响程度」筛选并汇总了五项最具参考意义的相关工作。其余工作（JuiceFS 作为实验主对比对象，CubeFS 作为技术路线参照，CFS 作为细粒度临界区分析的思想来源，Yang 等人的批量接口对应 readdirplus 实现，FetchBPF 与 FBMM 作为面向操作系统层的周边借鉴）均已在 前文展开，此处不再重复列出。

表 1-1 五项最具参考意义的 KV 元数据相关工作

工作	类别	对本课题的参考
TableFS ^[1]	KV 元数据	验证了 KV 存储用于元数据的可行性；暴露键编码、readdir 效率和事务缺失三个问题，本课题逐一改进
IndexFS ^[12]	KV 元数据	列式目录布局启发了 readdirplus 设计；其目录分片思路为后续元数据横向扩展提供参考
Mantle ^[2]	KV 元数据	三列族设计和 Delta Record 机制直接启发本课题的多列族架构与 DeltaOp 懒折叠方案；工业级验证增强可信度
SingularFS ^[19]	单机 MDS	说明元数据关键路径的细致优化本身就能明显抬高分布式文件系统的性能上限
CFS ^[20]	并发控制	细粒度临界区分析方法影响本课题的 PCC 事务设计，只锁涉及的 key 而非整个目录

上述工作各自解决了一部分问题；把它们组合到「Client + MetadataServer + DataServer + FUSE + RocksDB + 完整 POSIX」这一最小可用系统中，仍有若干问题需要回答。下一节先把这些空白点明，再给出 RucksFS 与它们的具体定位

对比。

1.2.4 现有研究不足与本课题定位

综合上述工作，有三个问题尚未在「分布式访问路径 + POSIX + 嵌入式 KV」这一完整组合下被系统处理。

其一，事务和并发控制这一环较为薄弱。TableFS^[1]所用的 LevelDB 没有事务语义，多进程写入只能在上层依靠粗粒度锁保护，单用户基准测试下问题不明显，但在 make 并行构建、rsync 同步、并发日志写入等真实负载下竞态会暴露出来。IndexFS^[12]引入 WriteBatch 保证单次操作的原子性，但 rename 同时涉及源目录和目标目录的多条记录，WriteBatch 并不覆盖跨操作的一致性约束，跨目录 rename 在 IndexFS 中仍需要上层自行加锁。RocksDB 自 6.0 版起已经提供了成熟的悲观事务（PessimisticTransactionDB），支持行级锁、GetForUpdate 和冲突检测；配合大端键编码的前缀局部性，还能把冲突范围限制在单个父目录内。在公开文献中，面向完整 POSIX 语义、围绕 RocksDB 悲观事务组织整套元数据操作的原型较少见。

其二，父目录 inode 的写放大问题长期被忽略。TableFS、JuiceFS、CubeFS 处理目录变更时基本采用读-改-写方式：create/unlink 需更新父目录的 mtime 和 ctime，mkdir/rmdir 还会更新父目录的 nlink。这些字段每次都被完整读出、修改后再写回，在热点目录下（例如存放了数十万文件的临时目录）会把所有写入串到同一条父 inode 记录上，LSM-tree 擅长的顺序追加特性反而无法发挥。Mantle^[2]的 Delta Record 是目前唯一在生产环境中系统性解决该问题的方案，但它依赖的 TafDB 分布式事务数据库和 IndexNode 缓存一致性协议整体较为复杂，不便直接引入本课题这种轻量的三段式架构。如何在 MetadataServer 本地的 RocksDB 上以较低复杂度实现类似的增量更新语义、同时保证 POSIX 的原子性和读一致性，公开可参考的方案相对较少。

其三，现有系统普遍偏向通用分布式可扩展性，对专用元数据服务器本身的执行路径关注不足。IndexFS、CubeFS、Tectonic 普遍引入 Raft 共识、一致性哈希、分区迁移等重型机制，在 EB 级规模下这些是必要的基础设施，但也会让原本可以在专用元数据服务器内部快速完成的简单目录操作退化为较重的事务

流程。SingularFS^[19] 在 RDMA 网络和持久内存上实现了单节点数百万级 ops/s 的元数据吞吐，说明元数据引擎自身经过关键路径优化后仍有较大空间。但「分布式部署 + 专用元数据引擎 + 完整 POSIX 合规」这一组合下可复现的公开原型相对较少：大规模工业系统多不开源，学术原型多以单机或简化 API 为界。

把上述三点合起来看，本课题的定位相对清晰：设计并实现一个基于 RocksDB 的分布式文件系统原型 RucksFS，由 FUSE Client、MetadataServer、DataServer 三部分组成，通过 gRPC 连接客户端与服务端；元数据侧以 RocksDB 为基础，采用多列族、悲观事务和前缀迭代器等特性，配合一套适合文件系统访问模式的键编码和增量更新机制。与已有工作的主要区别集中在三处：其一，通过 PCC 事务为所有目录变更操作统一提供原子性保证，而非依赖外部锁或仅使用 WriteBatch；其二，借鉴 Mantle 的 Delta Record 思路，用 DeltaOp 与懒折叠解决父目录 inode 写放大，但全部逻辑在 MetadataServer 本地 RocksDB 内完成，不引入外部通用分布式事务数据库和复杂的缓存一致性协议；其三，以 Rust 实现，基于 FUSE 异步运行时和 gRPC 通信构建完整的分布式访问路径，在不修改内核、不依赖特殊硬件的前提下验证该架构的语义和性能表现。表 1-2 从 KV 引擎、事务支持、增量更新、用户态接口这四个维度，将本课题与相关系统并列对比。

表 1-2 本课题与相关系统的定位对比

系统	KV 引擎	事务支持	增量更新	用户态接口
TableFS ^[11]	LevelDB	无	无	FUSE
IndexFS ^[12]	LevelDB	WriteBatch	无	自定义库
Mantle ^[2]	TafDB	分布式事务	Delta Record	对象存储 API
JuiceFS ^[9]	多种后端	依赖后端	无	FUSE
CubeFS ^[10]	内存 B-tree	Raft	无	FUSE
RucksFS (本课题)	RocksDB	PCC 事务	DeltaOp	FUSE

1.3 论文的主要内容与结构

1.3.1 主要工作

本课题围绕「基于 KV 存储的文件系统元数据管理」这一目标，主要完成了以下五项工作：

(1) 设计并实现基于 RocksDB 多列族的元数据存储方案。不同类型的元数据 (inode 属性、目录项、增量记录、系统元数据) 分别放在独立列族中，各自配置压缩策略和缓存参数。目录项列族的键采用「以边为中心」的编码：父 inode 号和子文件名按大端序拼接作为键，每条目录项对应目录树中一条父子边。文件操作由此转化为对边的增删：create 插入一条、unlink 删除一条、rename 删旧插新，均为常数时间。大端序编码使字节序与数值序一致，同一父目录下的子项在键空间中自然相邻，readdir 通过前缀扫描、lookup 通过点查即可完成。

(2) 引入增量更新与懒折叠 (lazy folding) 机制，并基于 RocksDB 悲观事务统一处理元数据并发。文件创建和删除不直接修改父目录 inode，而是以 DeltaOp 形式追加一条增量记录；读路径在 getattr 或 readdir 时将基值与增量在线合并后返回；后台线程在增量数量超过 32 条时将其折叠回基值，使高并发写入下 KV 层面的读-改-写次数明显减少。所有元数据变更操作 (create、mkdir、unlink、rmdir、rename、setattr 等) 通过 RocksDB 悲观事务完成加锁和冲突检测，以原子提交保证多键更新的一致性；事务冲突时服务端先自动重试最多 3 次，仍失败后在 FUSE 层映射为 EAGAIN 返回。

(3) 基于 FUSE 实现覆盖上述目录操作与 read/write 的完整 POSIX 文件操作集合。客户端基于 Rust 的 fuse3 库 (0.8 版，启用 tokio-runtime 与 unprivileged 特性) 与 tokio 异步运行时实现，按异步模型处理请求；挂载时启用 default_permissions 与 allow_other，常规权限检查交由内核 VFS 处理。

(4) 实现由 Client、MetadataServer、DataServer 三段组成的分布式部署形态，三者通过 gRPC (tonic) 通信。客户端内部设有一个请求分发层，把元数据请求交给 MetadataServer、数据请求交给 DataServer；对于 open (需要先从 MetadataServer 获取数据路由再落到 DataServer)、release (触发数据清理) 这类跨服务的语义，由客户端统一编排。

(5) 完成了两条线的系统评估。正确性方面，在分布式部署下运行 `pjdfstest` 套件共 398 条用例，按本文实现范围分解后在「实现范围内」用例上全部通过；性能方面，在 1 台 64 核服务器加最多 96 台 2 核客户端的集群上使用 `mdtest` 压测，中低并发区间 ($N \in \{2, 8, 32\}$) 将 RucksFS 与 JuiceFS+TiKV、NFS v4.2 作三方横向对比，高并发区间 ($N \in \{64, 96\}$) 改用 Delta 与 NoDelta 两个构建变体作内部对照，检验 DeltaOp 机制在共享父目录高请求速率下的作用。

上述实现的完整源代码、测试编排脚本、本文所用的原始测量数据以及本论文的 LaTeX 源码均公开在 <https://github.com/csjpgg/rucksfs>，按第 5 章的参数即可复现。

1.3.2 论文结构

本论文共六章，各章安排如下：

第一章绪论。交代研究背景，分析 KV 存储用于文件系统元数据时需要面对的技术挑战，梳理国内外相关工作，给出本课题的目标与定位。

第二章相关技术基础。介绍 FUSE 的工作原理与请求处理流程、RocksDB 的 LSM-tree 架构与列族/事务机制、POSIX 文件系统语义的几项关键约束以及悲观并发控制 (PCC) 的基本概念。

第三章元数据组织与事务模型。展开 MetadataServer 内部的设计：多列族的命名空间划分、目录项的大端序边键编码、DeltaOp 增量追加与懒折叠机制、PCC 事务封装与冲突重试策略，以及各 POSIX 目录操作在事务层面的具体落实。

第四章系统实现。从代码组织和进程协作的角度把第三章的设计落到一个完整的分布式原型：7 个 crate 的分层划分、客户端把 FUSE 请求分发到 MetadataServer 或 DataServer 的方式、`open` 等跨服务操作的编排流程、MetadataServer 与 DataServer 各自的实现、gRPC 接口与多通道连接池优化，以及跨服务的错误处理与崩溃恢复边界。

第五章测试与结果分析。用 `pjdfstest` 和 `mdtest` 两条线检验前文设计：前者在分布式部署下运行 398 条用例，按实现范围分解后给出合规性结果；后者在同一套集群上分别作横向对比与 Delta/NoDelta 内部对照，并解读数据形态背后的原因。

第六章总结与展望。回顾主要成果，讨论当前设计的局限，给出客户端缓存放宽、跨服务 RPC 合并以及 DataServer 完整化等方向的后续改进思路。

2 相关技术基础

RucksFS 的设计建立在三组基础技术之上：FUSE 负责将 Linux VFS 请求交给用户态客户端，RocksDB 提供适合元数据工作负载的持久化引擎和事务能力，POSIX 语义则规定了文件系统操作应当呈现的行为。本章只介绍与后续设计直接相关的部分，不对这三个主题逐一铺开。

2.1 FUSE 用户态文件系统框架

ext4、XFS、Btrfs 等内核态文件系统运行在内核态，调用路径短，但开发、调试和故障定位都需要直接面对内核环境。FUSE (Filesystem in Userspace)^[31] 采用了不同的思路：内核只保留请求转发、进程阻塞/唤醒、权限协作等基础机制，具体的文件系统逻辑放到用户空间实现。从 Linux 2.6.14 起 FUSE 被合并进主线内核，随后逐渐成为用户态文件系统的事实标准，JuiceFS^[9]、CubeFS^[10]、GlusterFS、SSHFS 等系统均通过 FUSE 对外提供 POSIX 接口。

本课题选择 FUSE 有两点原因。其一，FUSE 暴露的接口与 POSIX 操作几乎一一对应——lookup 对应路径解析、create 对应 open(O_CREAT)、rename 对应目录项移动——把「每一次 POSIX 调用在文件系统内做了什么」的过程清晰地展现在用户态；其二，用户态开发和调试比内核开发门槛低，不需要反复加载内核模块、也不容易因为一次崩溃拉垮整台机器，迭代速度更快。Linux VFS 继续负责系统调用入口和基础权限检查，用户态客户端专注于元数据事务、路由和与后端服务的交互。

2.1.1 工作机制

FUSE 在 RucksFS 中的请求通路可以抽象为表 2-1 中的 6 个阶段。表中重点在各段的边界与职责，而非具体函数名。

当应用对挂载点执行 open()、stat()、mkdir() 等系统调用时，请求首先进入 VFS。若目标路径位于 FUSE 挂载点下，VFS 不会继续调用 ext4 之类的内核文件系统实现，而是把请求转交给 FUSE 内核模块。该模块为请求分配请求 ID，封装操作码与参数，将请求写入 /dev/fuse，并让发起系统调用的线程阻塞等待。

表 2-1 FUSE 请求的分层执行路径

阶段	位置	动作
1	应用进程	调用 <code>open()</code> 、 <code>stat()</code> 、 <code>mkdir()</code> 等 POSIX 接口
2	Linux VFS	识别目标路径属于 FUSE 挂载点，并把请求转交 FUSE 内核模块
3	FUSE 内核模块	为请求分配 ID，封装 <code>opcode</code> 与参数，写入 <code>/dev/fuse</code> ，并阻塞原线程
4	FUSE 客户端	从 <code>/dev/fuse</code> 读取请求，解析协议头并派发到对应回调
5	RucksFS 用户态逻辑	通过 <code>VfsCore</code> 调用 <code>MetadataServer</code> 或 <code>DataSeter</code> ，得到处理结果
6	内核返回路径	守护进程写回响应，内核按请求 ID 唤醒原线程并返回系统调用结果

用户态守护进程在挂载时已经打开 `/dev/fuse`，随后持续从该设备读取请求帧。守护进程根据 `opcode` 识别请求类型，再把请求派发到开发者实现的回调，例如把 `LOOKUP` 派发给 `lookup()`，把 `CREATE` 派发给 `create()`。回调完成后，守护进程把结果编码为响应帧并写回 `/dev/fuse`；内核再据请求 ID 唤醒原先阻塞的线程，将结果接回系统调用返回路径。整条路径上，内核只负责请求的入口与调度，文件系统的语义和存储实现全部留在用户空间，双方唯一共享的交互媒介就是 `/dev/fuse`。

2.1.2 回调模型与异步实现

FUSE 暴露的接口是一组与 VFS 操作一一对应的回调。表 2-2 列出了本文直接涉及的核心回调。它们与 POSIX 操作基本一一对应：`lookup` 用于路径解析，`getattr` 对应 `stat()`，`create` 对应 `open(O_CREAT)`，`rename` 对应目录项移动，`readdir` 对应目录遍历。

在实现机制上，RucksFS 的客户端使用 Rust 的 `fuse3`^[32] (0.8 版，启用

表 2-2 FUSE 核心回调接口（本文涉及部分）

回调	对应 VFS 操作	语义
lookup	路径解析	给定父 inode 和文件名，返回子 inode 的属性
getattr	stat()	返回 inode 的元数据（mode、size、mtime 等）
setattr	chmod/chown/utimes/truncate	修改 inode 属性
create	open(O_CREAT)	在父目录下创建文件并返回文件句柄
open	open()	为已有文件分配 file handle
release	close()	释放 file handle 及相关资源
mkdir	mkdir()	在父目录下创建子目录
unlink	unlink()	从父目录中删除文件的目录项
rmdir	rmdir()	删除空目录
rename	rename()	将目录项从源位置移动到目标位置
readdir	getdents()	列出目录下的所有条目
read	read()	读取文件数据
write	write()	写入文件数据
fsync	fsync()	将文件数据与元数据刷盘

tokio-runtime 与 unprivileged 特性)。fuse3 是围绕 tokio 异步运行时封装的 FUSE 绑定，回调以异步函数的形式提供。一次请求的处理流程可以概括为：运行时从 `/dev/fuse` 读到请求后，把它派发到对应的异步回调中作为一个协程执行；当协程需要等待元数据 RPC、数据 RPC 或其他异步 I/O 完成时，运行时会让该协程暂时让出线程，转去推进其他已经就绪的协程，待所等待的事件完成后再把原协程恢复。整个过程中，实际占用的操作系统线程数量不会随并发请求数线性增长。

对 RucksFS 这种客户端与服务端分离的架构而言，这种执行方式相当合适。FUSE 层本身只是收发请求，真正耗时的是用户态客户端与 MetadataServer、DataServer 之间的网络往返；采用协程 + 阻塞等待的写法后，一个协程挂起在 gRPC 调用上时，运行时可以立刻去处理下一个 `lookup`、`getattr` 或 `readdir`。这种模式在网络 I/O 占主导的元数据密集负载下可以明显提高并发度，同时把并发编排交给运行时，上层代码保持顺序控制流，可读性和出错率与同步写法相差不多。

2.1.3 性能开销特征

FUSE 的主要代价来自跨内核态与用户态边界的固定成本。对 ext4 这类内核态文件系统，一次 `stat()` 基本在内核内完成；而在 FUSE 文件系统上，同一次 `stat()` 需要走完「应用线程进入内核、请求写入 `/dev/fuse`、守护进程读取请求、守护进程写回响应、内核唤醒原线程」这一整套。多出的开销主要有两类：上下文切换，以及请求/响应两帧在 `/dev/fuse` 上的拷贝。

Vangoor 等人^[33]在 FAST'17 上报告了针对 FUSE 的一组性能测量：元数据类型操作（`lookup`、`getattr` 等）在 FUSE 文件系统上相对内核态文件系统下降约 20%–60%；而在顺序大文件读写场景下，这一下降幅度缩小到 5%–20%。两者差距的原因是：元数据操作本身执行时间短，跨边界的固定开销在单次延迟中占比高；一旦磁盘或网络 I/O 成为主导，这些固定成本会被单次 I/O 的时间摊薄，占比随之降低。

对 RucksFS 这种分布式原型而言，FUSE 并不是主要瓶颈。决定整体性能的是客户端到 MetadataServer 的 gRPC 网络路径——每次文件操作至少包含一次跨

节点 RPC 往返，其延迟远大于 FUSE 自身的内核-用户态边界开销。因此 FUSE 引入的是一个常数级别的固定开销，而不是会随规模劣化的瓶颈。

2.2 RocksDB 存储引擎

RocksDB^[5] 是 Meta 在 LevelDB^[4] 基础上发展出的嵌入式键值存储引擎，针对 SSD 场景和高写入负载做了较多工程优化，已被 TiKV、MyRocks、CockroachDB 等上层系统广泛用作持久化后端。对「小值、高频、随机写密集」的文件系统元数据负载而言，RocksDB 的 LSM-tree、列族、前缀布隆过滤器和事务接口都是较为合适的工具。

2.2.1 LSM-tree 数据结构

LSM-tree (Log-Structured Merge-tree) 最早由 O’Neil 等人提出^[3]，其核心思想是把随机写换成顺序写。新写入先进入内存中的 MemTable，同时追加一条 WAL；MemTable 写满后作为一整块有序 SST 文件刷盘；后台 Compaction 线程按键范围合并重写各层 SST，以维持读性能和空间利用率。图 2-1 展示了这一流程。

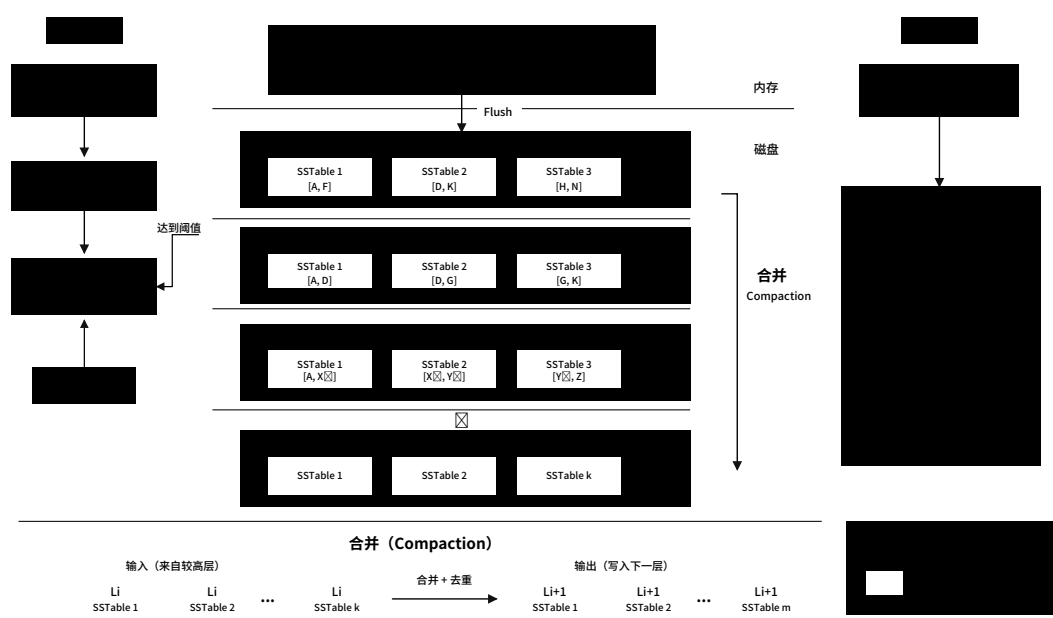


图 2-1 LSM-tree 的写入与 Compaction 流程

图中左侧是写入路径：新数据先进入 MemTable，达到阈值后作为一块 SST

文件刷盘；右侧是后台 Compaction：各层 SST 文件按键范围合并重写，消除过期版本并将数据有序下沉。这一思路在更早的日志结构文件系统^[34]中已有雏形，之后被 Bigtable^[35]、LevelDB^[4] 等工业系统逐步发展为面向 SSD 的高写入吞吐存储引擎。

对文件系统元数据而言，这一结构有两点直接影响。其一，单条目录项或 inode 属性的更新可以先在 MemTable 中完成一次内存追加，而不像 B 树那样立即修改磁盘页。其二，后台合并阶段会引入额外的写放大，因此键的设计和值的大小需要保持克制。WiscKey^[36] 的做法是把 value 与 key 分离存储，在此基础上降低 Compaction 时的数据搬移成本；RucksFS 则沿另一条路径——目录项和 inode value 都尽量保持小尺寸（第 3.2 节所述编码下分别为 12 字节和 57 字节），文件数据完全绕开 RocksDB，避免大 value 对 Compaction 的影响。

RocksDB 的读取路径则自上而下：先查活跃 MemTable，再查不可变 MemTable，然后自 Level 0 开始逐层搜索 SST 文件。为了降低点查代价，SST 文件内部维护索引块和 Bloom 过滤器；为了减少磁盘访问，常用数据块会进入块缓存。这也是文件系统元数据需要强调键的前缀局部性和点查友好性的原因：否则每次读取都会放大到多层 SST，产生明显的读放大。

2.2.2 Column Family 机制

Column Family（列族）是 RocksDB 较为实用的一项特性。它允许在同一个数据库实例内维护多组逻辑上隔离的键空间，各自拥有独立的 MemTable、SST 集合和压缩配置，但共享 WAL。对文件系统而言，这意味着 inode 基础属性、目录项、增量日志、系统元信息不必在同一种访问模式下被统一处理。

RucksFS 的元数据层正是依赖这一特性进行分层配置：inodes 列族承担以点查为主的 inode 属性访问；dir_entries 列族承担按父目录前缀扫描的目录项访问；delta_entries 列族承担高频的增量追加和后台折叠；system 列族仅存放 inode 分配器等低频元信息。拆分之后，不同列族可以分别配置 Bloom 过滤器、前缀提取器和压缩策略，而无需在一个全局配置上折中。

这种做法也有代价。列族数量增加后，内存中的 MemTable 和后台 Compaction 状态也相应增加；跨列族的一致性更新需要依靠事务或批量原子

提交来保证。具体的列族划分和跨列族一致性策略属于 RucksFS 元数据模型的设计内容，在第 3.2 节展开。

2.2.3 WriteBatch、事务与并发控制

RocksDB 在一致性相关的能力上分两层。底层是 WriteBatch，能够保证一组 put/delete 原子提交；上层是 TransactionDB，除原子提交外还提供 get_for_update、锁等待超时、冲突检测等事务能力^[37]。对普通 KV 业务，WriteBatch 通常已足够；但文件系统元数据的目录操作涉及「条件检查 + 多键修改」的组合，WriteBatch 只能保证后半段的原子性，对前半段的 TOCTOU 窗口无能为力。

TransactionDB 的做法是在读取键时通过 get_for_update 顺便获取排他锁，把后续可能的修改提前锁定在当前事务上下文内。锁按行（即 key）粒度持有，冲突范围仅限于被实际触及的键；结合大端编码下同一父目录下子项在键空间中的前缀相邻性，多数情形下的冲突只会发生在同一个目录的并发操作之间，目录之间互不干扰。锁等待超过阈值时会返回冲突错误码，由上层决定重试策略。

RucksFS 对目录操作的设计正是建立在这组能力之上：条件检查和多键修改被收拢在一次事务内，冲突通过行级锁解决，跨 inode 的加锁顺序按 inode 编号统一排列以避免死锁。对应的事务封装与重试策略在第 3.4 节给出。

2.2.4 Bloom 过滤器与前缀迭代器

Bloom 过滤器用于快速判断某个键是否「不存在于某个 SST 文件中」，从而避免无谓的磁盘访问；前缀迭代器则要求一组具有共同前缀的键在字节序上连续排列，以便对同一前缀做局部扫描。对文件系统目录项而言，这两个特性组合起来较为关键。

若把目录项键设计为「父 inode + 文件名」的拼接形式，同一父目录下的全部子项就天然共享固定前缀。这样一来，lookup(parent, name) 可以退化为点查，readdir(parent) 则可以退化为前缀扫描。再配合大端序编码，使 inode 的数值顺序与字节序保持一致，前缀范围查询在 RocksDB 中可以直接依赖字节比较器实现。

2.3 POSIX 文件系统语义

一个文件系统只要对外暴露 POSIX 接口，就需要遵守一整套关于 inode、目录结构、权限位和错误码的约定，不能只关注「数据能否读写」。本文并不追求实现 POSIX 的每个细节，而是聚焦于普通文件、目录和常用元数据操作这几部分，使其与 Linux 用户空间程序的预期尽量一致。

2.3.1 inode 模型与目录结构

POSIX 文件系统以 inode 作为对象的持久标识。目录并不是一组路径字符串的集合，而是「名字到 inode」的映射表。路径解析实质上是从根目录开始逐级查询目录项，再根据目录项获取下一层 inode。普通文件、目录、符号链接共用同一种基本身份表示，只是 mode、nlink 和数据解释方式不同。

这对存储层设计有两点直接影响。其一，路径树不能直接存为「全路径字符串到内容」的映射，否则 rename 就会退化为对整棵子树的递归重写；其二，目录项和 inode 属性需要在存储上分开，否则 lookup、readdir、getattr 都难以高效实现。

2.3.2 核心文件操作语义

create、mkdir、unlink、rmdir 和 rename 是本文最关心的 POSIX 操作。它们共同的难点不在于「写几条记录」，而在于必须同时满足若干一致性条件。表 2-3 对这些操作的核心约束做了归纳。

首先，目录项相关操作必须保持名字唯一。若同一父目录下已存在名为 foo 的目录项，再次 create(parent, "foo") 必须返回 EEXIST，而不能出现两个同名条目。其次，rmdir 只能删除空目录；若目标目录仍包含子项，必须返回 ENOTEMPTY。再次，rename 要求原子可见：外部观察者不应看到「旧名字已删除、但新名字尚未建立」的中间状态。

时间戳和链接计数同样属于语义的一部分。普通文件的 create 和 unlink 会更新父目录的 mtime/ctime；mkdir 和 rmdir 除更新时间戳外，还会改变父目录的 nlink。这些字段在高并发写入下会成为热点——许多共享父目录下的创建/删除都会指向同一条父 inode 记录，若以读-改-写方式更新，并发度会被该字

表 2-3 核心 POSIX 目录操作的语义约束

操作	典型返回	关键语义约束
create	成功或 EEXIST	同一父目录下名字必须唯一；普通文件创建会更新父目录的 mtime/ctime
mkdir	成功或 EEXIST	新建对象类型为目录；除更新时间戳外，还会使父目录 nlink 增加 1
unlink	成功或 ENOENT/EISDIR	删除的是普通文件目录项；会更新父目录时间戳；若文件仍被打开，则目录项删除与对象回收需要分离
rmdir	成功或 ENOTEMPTY	只能删除空目录；成功后父目录 nlink 减少 1，并更新 mtime/ctime
rename	成功或相关错误码	源目录项删除与目标目录项建立必须原子可见；跨父目录移动目录时需要同步调整新旧父目录 nlink

段的锁争用所限制。

最后，打开文件与删除目录项之间的关系不能被忽略。POSIX 允许一个文件在目录项被移除后继续被已打开的句柄访问，直到最后一个句柄关闭为止。这要求元数据层区分「名字是否还在目录里」和「对象是否还能被已有句柄引用」两个状态：在 nlink 降到 0 但仍存在打开句柄时，只能删除目录项而不能立刻回收对象。

2.3.3 权限模型

POSIX 权限模型由 uid、gid 和 mode 三部分组成。FUSE 文件系统既可以在用户态自行实现访问控制，也可以把常规权限判断交给 Linux VFS。RucksFS 采用后一种方式：挂载时启用 default_permissions，由内核在请求进入用户态之前完成基础权限检查。这样做的好处是，常规的 owner/group/other 语义不必在 FUSE 守护进程中重复实现；用户态可以更专注于元数据事务和存储路径本身。

这并不意味着元数据层可以忽略权限字段。文件创建和目录创建仍然必须

正确记录 `uid`、`gid` 和 `mode`，否则内核后续的权限判断就失去依据。因此客户端在处理 `create` 和 `mkdir` 时会把请求发起者的身份信息一并传递给元数据服务。

2.4 本章小结

本章介绍了 RucksFS 依赖的三项基础。FUSE 提供了把 Linux VFS 请求交到用户态客户端的标准通道，以及一套与 POSIX 操作对齐的异步回调接口；RocksDB 用 LSM-tree、列族和悲观事务这三件事分别应对元数据的写放大、访问模式分化和并发正确性问题；POSIX 则规定了目录操作、链接计数、时间戳和权限模型的行为规范，并给出了热点父目录、删除与打开句柄等几处需要特别处理的语义约束。下一章在此基础上进入 RucksFS 自身的元数据组织与事务模型。

3 元数据组织与事务模型

本章集中讨论 MetadataServer 内部的核心设计。内容分为三条线：把目录树映射到 RocksDB 键空间的数据模型、针对父目录写热点的 DeltaOp 增量更新机制，以及保证并发目录操作正确性的悲观事务封装。最后用 create、unlink、rmdir、rename、readdir 等核心操作把前三条线的设计串起来，说明一次 POSIX 目录操作在本文的实现中具体如何完成。

3.1 设计目标

讨论具体实现之前，先把本章要回答的几项问题说清楚。这几项目标贯穿后续各节的设计取舍。

第一，对外保留标准的 POSIX 语义。应用程序通过 open、rename、unlink 等系统调用访问挂载点，不为 RucksFS 改写任何代码。接口选择 FUSE 即出于此。

第二，目录树到 RocksDB 键空间的映射需要同时支持 lookup、readdir 和 rename 三类访问模式。lookup 要点查快，readdir 要前缀扫描连续，rename 要能在常数步内完成、不触发子树级联改写。这三项约束共同决定了键编码方案。

第三，并发目录操作的正确性要由存储层事务保证。名字唯一、目录非空判断、跨目录重命名的原子性、延迟删除这几类语义，不能依赖“多数情况下不会冲突”的经验假设；必须把检查和修改收敛在同一事务内。这是采用 RocksDB 悲观事务的直接动机。

第四，父目录属性的高频更新是需要单独处理的热点。mtime、ctime、nlink 每次目录修改都会动到，朴素的读-改-写在共享目录下会反复冲突。DeltaOp 增量更新机制就是面向这一热点设计的。

3.2 元数据存储模型

本节分三步回答“数据怎么组织在 RocksDB 里”：一是目录树如何映射为 KV 记录，二是不同访问模式的记录如何按列族隔离，三是单条记录在字节层面怎么编码。

3.2.1 目录树到键空间的映射

RucksFS 的命名空间是一棵目录树，但 RocksDB 只看字节串到字节串的映射。真正要持久化的对象只有两类：inode 描述文件系统对象本体，记录类型、权限、大小、链接计数、时间戳等；目录项描述目录树上的一条父子边，记录“某个父目录下的某个名字指向哪个 inode”。路径本身不存，而是由一连串目录项逐级拼起来。以 `/docs/paper.tex` 为例，先在根目录下查 `docs` 对应的目录项拿到目录 inode，再在该目录下查 `paper.tex` 对应的目录项，最终定位到目标 inode。图 3-1 用一张具体例子说明这种映射。

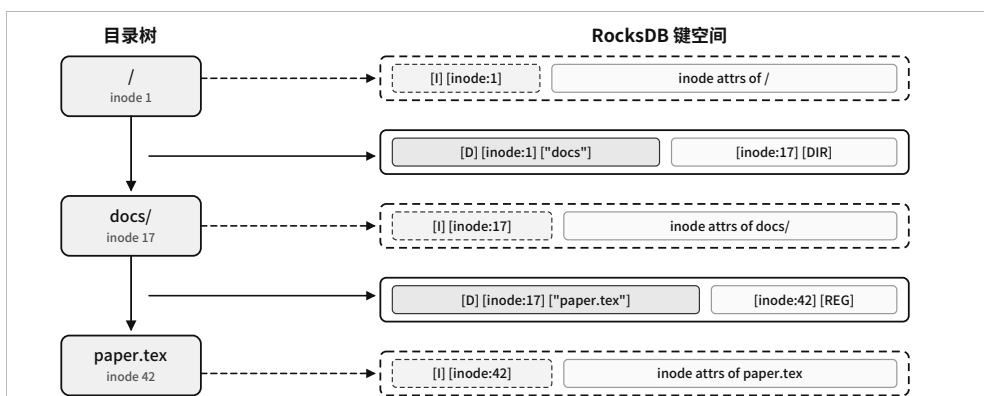


图 3-1 路径树到 RocksDB 键空间的映射示意

图的左侧是路径 `/docs/paper.tex` 对应的目录树，右侧是存入 RocksDB 的 KV 记录。两者之间并非一一对应——每条父子边额外在键空间中占一条目录项记录，每个 inode 本身再占一条属性记录。路径中的每一步解析都只触及一条目录项和一条 inode 记录，与树的深度无关。

这种“以边为中心”的建模方式解决了一个具体问题：跨目录 `rename` 应当是常数代价。如果把整条路径作为键，移动一棵子树就要把子树内每一条路径记录都改写一次。以边为键则只需删除旧的父子边、插入新的父子边，与子树规模无关。其它两类访问也在这套建模下自然落位：`lookup` 是对单条目录项键的点查；`readdir` 是对“父 inode”前缀的范围扫描。

键设计的一个关键细节是编码顺序。RocksDB 按字节序比较键，因此若希望“同一父目录下的所有目录项在键空间中物理相邻”，inode 编号的字节序必须与数值序一致。2013 年的 TableFS^[1] 采用原生字节序——在小端机器上，inode 编号 256 对应的字节序表示和 257 在字节序上可能隔着大量其它 inode 的键，前

级扫描不得不在拿到结果后再按父 inode 过滤一次。RucksFS 统一使用大端序编码，使字节序顺序与数值顺序一致；同一父目录下的所有子项在 LSM-tree 中天然连续排列，readdir 可以直接依赖前缀迭代器和前缀 Bloom 过滤器完成扫描。

3.2.2 按访问模式划分的列族

下一个问题是：把全部元数据放在一个统一键空间里够不够？本文的做法是把 RocksDB 实例拆成四个列族——inodes、dir_entries、delta_entries、system——每个列族承担一类差别较大的访问模式。列族之间各自拥有独立的 MemTable、SSTable 层和压缩策略，但共享 WAL，保证跨列族的原子提交仍然可用。图 3-2 给出四个列族各自保存哪些记录，以及对应的键值格式。



图 3-2 RucksFS 元数据记录与列族布局总览

从图中可以看到四类记录的访问模式差别较大，对应的配置需求也不同，逐一说明如下。

inodes 列族承担 inode 属性、数据位置和符号链接目标三类记录的点查负载。这三类记录语义上并不完全同类，但都按 inode 直接定位。按同一种访问模式放在一起，省去为少量辅助记录单独维护一套 MemTable 与压缩状态的开销。该列族启用点查 Bloom 过滤器。

dir_entries 列族承担命名空间的边关系。lookup 按”父 inode + 名字”点查单条，readdir 按父目录枚举全部子项，这两种访问都依赖前缀局部性。该列

族启用 9 字节的前缀提取器（单字节前缀加 8 字节父 inode 编号），使同一父目录下的目录项按前缀连续排列，前缀 Bloom 过滤器也因此可以生效。

`delta_entries` 列族承担热点目录的增量写入。父目录的 `mtime`、`ctime` 和 `nlink` 变化不走“覆盖写回基值”这条路径，而是以 `DeltaOp` 形式追加到这里。该列族上的记录短小、生命周期短，由后台线程定期折叠消费，因此刻意关闭了压缩，省去热路径上的编解码开销。具体的追加与折叠流程在 3.3 节展开。

`system` 列族仅存放 inode 分配器等低频系统状态。访问频率远低于其它列族，但仍需与普通元数据一起走原子提交，因此留在同一个 RocksDB 实例内。单独隔离是为了避免它与热点列族共用压缩和缓存策略。

这套划分不按代码模块切分，而是按访问模式切分。每个列族可以独立调参，互不干扰；跨列族的原子提交则由 RocksDB 事务兜底。

3.2.3 值的二进制编码

列族边界确定之后，单条记录的字节编码是最后一层。RucksFS 不使用通用序列化框架，而是为 `InodeValue` 和目录项值都采用手工布局的定长编码，目的是让 RocksDB 读到数据后可以按固定偏移直接切片，不必承担变长解析的开销；调试时也可以从二进制转储里直接读出字段。

`InodeValue` 共 57 字节。首字节留作格式版本号，之后依次排列 inode 编号（8 字节）、文件大小（8 字节）、模式位（4 字节）、链接计数（4 字节）、uid 与 gid（各 4 字节）、以及三类时间戳（`atime`、`mtime`、`ctime` 各 8 字节）。版本号的作用是在未来真需要扩展字段时提供兼容通道，不必推翻整套键空间。

目录项 value 固定为 12 字节：`[child_inode: u64 BE][child_mode: u32 BE]`。把子 inode 编号与模式位存在同一条目录项 value 里有一项直接好处——`readdir` 在遍历目录时即可判断每个条目是普通文件、目录还是符号链接，不必为每个条目再回查一次 `inodes` 列族。这对目录下有大量条目的场景（例如构建产物目录）意义直接。

增量记录 value 使用 `DeltaOp` 编码，下一节展开。

3.3 DeltaOp 增量更新与懒折叠

POSIX 语义要求目录项变动时同步更新父目录的 mtime/ctime, mkdir/rmdir 还要动 nlink。朴素做法是每次操作都读出父 inode 旧值、改几个字段、写回同一条键。这条路径在共享父目录高并发下有两个问题：其一，RocksDB 事务会把所有并发操作都阻塞到同一条 inode 键的行锁上；其二，即便事务能排队通过，LSM-tree 层面也会在同一条键上堆出大量版本，compaction 反复归并，写放大抬升。/tmp、CI 产物目录、模型训练的 checkpoint 目录都属于这类热点。

DeltaOp 的想法是把父目录属性更新从“覆盖基值”改成“追加增量”。每次目录变动只向 delta_entries 列族追加一条 DeltaOp，基值不动；读取时把基值与未折叠的增量合并后返回；后台线程在增量积累到阈值时再把它们折叠回基值。这一机制借鉴了 Mantle^[2] 的 Delta Record 思路，但本文的实现全部限制在单个 RocksDB 实例内完成，不需要外部分布式事务数据库或跨服务缓存一致性协议。图 3-3 展示这三条路径——写、读、后台折叠——在同一组列族上如何协同。

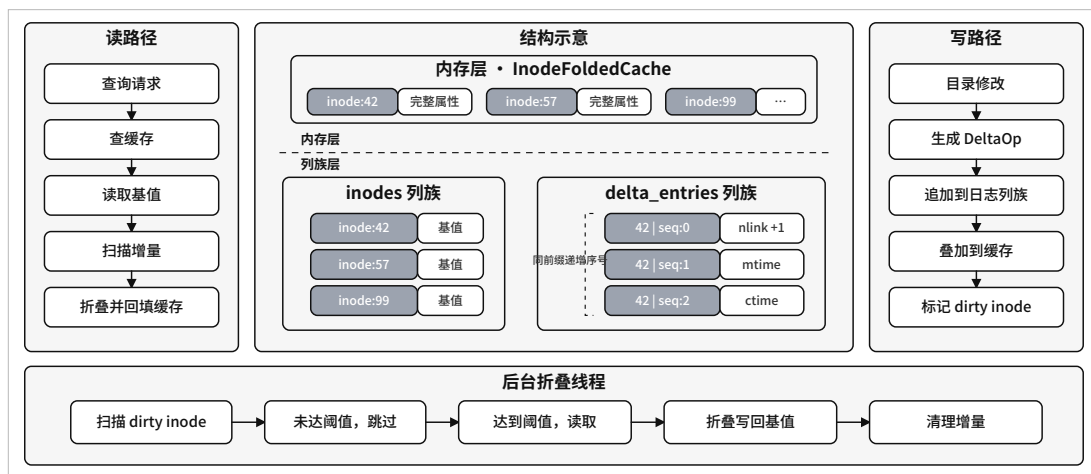


图 3-3 DeltaOp 的写路径、读路径与后台折叠线程

图中的中间部分是两层存储：上方是内存中的 InodeFoldedCache，下方是 RocksDB 的 inodes 和 delta_entries 两个列族。左侧的读路径从缓存开始；右侧的写路径仅向 delta_entries 追加；下方的后台折叠线程周期扫描 dirty inode，把积累的增量折叠回基值。下文分三段展开。

写路径。 当 create、unlink、mkdir、rmdir、rename 改动到父目录属性时，事务内部不修改 inodes 中父 inode 的基值，而是向 delta_entries 追加一

条 DeltaOp，键的格式为 [X][inode: u64 BE][seq: u64 BE]。序号 seq 单调递增，保证每次写入的目标键互不相同；不同事务即使同时修改同一父目录，也落在不同的键上，不争抢行锁。增量类型目前有四种：IncrementNlink 用于子目录的增删，SetMtime、SetCtime、SetAtime 分别对应三类时间戳的绝对值更新。这里把时间戳设计成绝对值而非增量，是因为同一次操作只会产生最新时间戳，折叠时按 seq 取最后一条即可；nlink 则是真正的累加量，折叠时要把所有 IncrementNlink 的增量相加。同一条事务提交时会一次性标记对应的父 inode 为 dirty。

读路径。getattr 或 readdir 需要拿到父目录最新属性时，按三步推进：首先查 InodeFoldedCache（一个 16 分片的内存 LRU，默认总容量 10 000 条；键为 inode 编号，值为已经折叠好的 InodeValue），命中则直接返回；若未命中，则从 inodes 读出基值，再按前缀扫描 delta_entries 得到所有未折叠的增量，在内存中按 seq 顺序把增量叠加到基值上；最后把叠加结果写回 InodeFoldedCache，供后续请求直接命中。缓存的 16 分片按 inode 编号做 Fibonacci 哈希分派，分片内各自持有一把细粒度锁，并发访问时锁冲突面积远小于一把全局锁。

后台折叠。写路径不写基值就要付出读路径叠加的代价，因此单个 inode 上的 DeltaOp 数量不能无限增长。后台的折叠线程周期扫描 dirty inode 列表（扫描周期 5 秒），当某个 inode 下的增量数量超过阈值（默认 32 条）时触发折叠：读出基值，按 seq 顺序叠加增量得到新基值，在一次事务内把新基值写回 inodes、同时把已消费的增量从 delta_entries 中删除。折叠结果也会注入 InodeFoldedCache，使下一次读取能够命中。折叠本身是一次写事务，但并不出现在 POSIX 调用的主路径上，不影响用户请求延迟。

这一机制的收益只在高并发的共享父目录场景下出现。低并发下，朴素的读-改-写与 DeltaOp 在吞吐上相差不大，因为前者并不触发事务冲突。第 5.3.6 节通过 Delta 与 NoDelta 两种编译变体对照——开关由 batch_parent_deltas 这一 cargo 特性控制，其余代码完全一致——直接衡量这一机制在不同并发下的效果。

DeltaOp 的代价也需要正面说明。其一，读路径引入了叠加动作，缓存未命中时会多一次前缀扫描。InodeFoldedCache 和 32 条折叠阈值两项共同控制这部分成本：缓存命中时完全绕过叠加；即使未命中，单 inode 的增量数量也有上

限。其二，增量记录消耗额外存储。但 DeltaOp 单条很小，delta_entries 又不启用压缩，净存储占用可以忽略。

3.4 悲观事务与并发正确性

元数据操作的正确性不只来自“写下了什么”，还来自“什么时候写、和哪些读共享视图”。本节集中讨论事务边界、加锁顺序与冲突重试。具体到每个 POSIX 操作上的表现放在 3.5 节。

3.4.1 原子提交的封装

RucksFS 的目录操作几乎都不是单键更新。create 至少同时写入目录项、目标 inode、数据位置以及父目录的增量；rename 可能同时改动源目录项、目标目录项、被移动对象以及潜在被覆盖对象。事务模型首先要回答的问题是：哪些修改必须作为一个不可分割的整体提交？

本文的做法是把 RocksDB 的原生事务接口封装起来，对外提供一个更抽象的 AtomicWriteBatch。上层调用方通过 StorageBundle::begin_write() 拿到一个事务上下文，向其中追加跨列族的写入操作，最后统一 commit。底层最终调用 RocksDB TransactionDB 的提交接口，把跨 inodes、dir_entries、delta_entries 的修改作为一次原子操作持久化。这样上层代码可以按“目录项是否存在”、“应该写哪些 inode 记录”、“父目录需要追加哪些增量”组织动作，不必显式处理不同列族的底层细节。

3.4.2 PCC 与 TOCTOU 窗口

原子提交只保证“一组写入同时生效”，并不防止“读到过期数据再据此决定写什么”。create、rmdir、rename 都存在这一问题：若先在事务外检查目标名字是否已存在、目标目录是否为空，再进入事务执行修改，检查结果在事务开启之前就可能失效。

本文用 RocksDB 悲观事务 TransactionDB 的 get_for_update 解决这个问题。它在读取键的同时获取该键上的排他锁，使“检查条件”和“执行修改”共享同一个事务视图内。只有持有该行锁的事务能看到并修改对应键；其它事务看到的要么是该行锁释放前的旧视图、要么在事务提交后看到新值，不存在中间状态。行级锁把冲突面积限制在被实际触及的键上——RocksDB 的行级锁本身就

按 key 粒度持有，结合本文在 `dir_entries` 列族上的大端前缀局部性，`create` 等操作的锁冲突只发生在同一个父目录的并发操作之间，跨目录并发互不干扰。

3.4.3 跨对象事务的加锁顺序

跨多个 inode 的事务需要面对死锁风险。例如两个 `rename` 事务若各自先锁源目录再锁目标目录，且双方的源和目标互换，就会形成循环等待。解决这一问题的标准做法是统一加锁顺序。本文的做法是：凡可能涉及多个 inode 的事务，都先把相关 inode 编号收集起来并排序，再按升序依次 `get_for_update`。图 3-4 以两个并发 `rename` 事务为例说明这一顺序的作用。

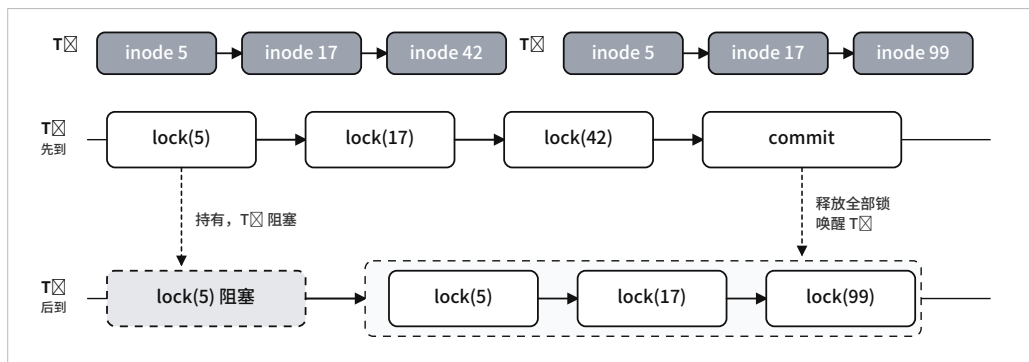


图 3-4 rename 事务按 inode 编号升序加锁避免死锁

图中 T_1 和 T_2 是两个并发事务，各自涉及三个 inode。 T_1 先到，按升序锁 5、17、42； T_2 后到，想锁 5，发现 5 已经被 T_1 持有，于是在 5 上阻塞。 T_1 提交后释放全部锁， T_2 随即按同样的升序拿到 5、17、99 三把锁。两个事务的锁等待图始终是无环的，不会形成死锁。

3.4.4 锁等待与冲突重试

即使加锁顺序统一，瞬时冲突仍不可避免——例如两个事务对同一目录并发 `create`，必有一个需要等另一个释放锁。本文为锁等待设置了 5 秒超时。超时后 RocksDB 返回 `TransactionConflict`，但这并不直接抛给用户：MetadataServer 在服务端有限重试，最多 3 次，起始退避 50 μ s，每次翻倍；仅在重试仍失败时，才向上返回 `TransactionConflict`，最终在 FUSE 层映射为 `EAGAIN`。这样偶发冲突不会转化为用户可见错误；`EAGAIN` 反映的是“事务冲突暂时无法消解”这件具体的事，而非任意 I/O 异常。

3.5 核心操作实现

本节按“POSIX 语义 → 事务流程 → 并发与性能考量”的顺序，对几项核心 POSIX 操作做统一的说明。重点不是罗列代码，而是让读者看到前面讨论的键编码、事务模型与 DeltaOp 在每个操作上具体如何组合起来。

3.5.1 create 与 mkdir

create 的难点不在于“写入一个 inode”，而在于需要同时满足三件事：目标名字在此前不存在、新对象以完整状态对外可见、父目录的属性更新与前两者原子一致。若拆成多步独立执行，就容易出现“inode 已建立但目录项尚未插入”或“同名检查通过但已被抢插”这种中间状态。算法 3-1 给出当前实现的 create 流程。

算法 3-1 create(parent, name, mode, uid, gid) 的核心流程

- 1: 开启事务 batch
 - 2: 对目录项键 D[parent, name] 执行 get_for_update
 - 3: **if** 目录项已存在 **then**
 - 4: 返回 EEXIST
 - 5: **end if**
 - 6: 分配新 inode 编号，构造普通文件的 InodeValue
 - 7: 向 inodes 列族写入新 inode 基值
 - 8: 向 dir_entries 列族写入父目录到新文件的映射
 - 9: 向 inodes 列族写入该 inode 的 DataLocation
 - 10: 向 delta_entries 追加父目录的 SetMtime 与 SetCtime
 - 11: 提交事务
 - 12: 更新本地缓存并返回新文件属性
-

整条流程体现了前三节设计的协同。先用 get_for_update 锁住目标目录项键，把“同名检查”纳入事务视图；新对象的可见性由“一条新目录项 + 一条新 inode 记录 + 一条新数据位置记录”三者共同决定，全部在同一批次内写入；父目录属性走 delta_entries 而非覆盖 inodes。后一点在热点目录下尤其重要——若走覆盖路径，所有并发 create 都会争抢父 inode 的行锁；走增量路径之后，每次 create 写入的是独立 DeltaOp 键，彼此不冲突。

`mkdir` 与 `create` 共用同一条事务骨架，区别只在于：新对象类型位是目录、初始 `nlink` 为 2；父目录除了更新时间戳之外，还要追加一条 `IncrementNlink(+1)`，因为多了一个子目录。普通文件创建不会改变父目录 `nlink`，而目录创建会——这一差异看似细微，但直接决定了后续 `rmdir` 和 `rename` 如何维护父目录状态。

3.5.2 `unlink`、`rmdir` 与延迟删除

`unlink` 的核心流程：锁定目标目录项、确认类型、删除目录项记录、将目标 `inode` 的 `nlink` 减一、向父目录追加 `SetMtime` 与 `SetCtime`。关键是目录项删除与 `nlink` 更新处于同一事务，不会出现“名字已消失但 `nlink` 尚未归零”的中间视图。

`rmdir` 的难点在空目录检查。若先在事务外扫描一次目录、确认为空、再进入事务删除，扫描结果可能在事务开启前就因其他 `create` 而失效。本文的做法是把空目录判断本身也放入事务：在事务视图下对 `dir_entries` 做一次以目标目录为前缀的扫描，发现任意子项即返回 `ENOTEMPTY`。确认为空后，删除目标目录项与目标 `inode`，向父目录追加 `IncrementNlink(-1)` 和两条时间戳增量。整个流程没有依赖上层约定，空目录约束完全由事务保证。

删除语义还有一处 POSIX 细节——打开文件的延迟回收。已经 `unlink` 的文件，如果仍有进程持有打开句柄，应该继续可读可写，直到最后一个句柄关闭才真正清理。`unlink` 事务只做两件事：把目录项从命名空间中移除，把 `nlink` 减到零；若此时还有打开句柄，目标 `inode` 不立刻删除，而是进入 `pending_deletes` 列表。打开句柄由 `MetadataServer` 内部的 `open_handles` 维护，`open` 成功时对应 `inode` 的计数加一，`release` 时减一。当 `release` 把计数减到零且该 `inode` 在 `pending_deletes` 中时，服务端回收 `inode` 记录，同时把 `inode` 编号返回给客户端，由客户端到 `DataServer` 侧真正清理数据。”名字可见性 → 元数据可达性 → 物理回收”由此被分成三个阶段，和 POSIX 语义对齐。

3.5.3 `rename`

`rename` 是本章设计取舍最集中的操作。它同时依赖“以边为中心”的键建模、统一的加锁顺序、覆盖删除语义以及父目录属性增量。涉及的状态可能包括

源目录、目标目录、被移动对象、潜在被覆盖对象四类；整个过程还必须对外原子可见。算法 3-2 给出主流程。

算法 3-2 `rename(old_parent, old_name, new_parent, new_name)` 的核心流程

- 1: 读取并锁定源目录项
 - 2: 若目标目录项存在，则读取其 inode 信息
 - 3: 收集全部相关 inode 编号并排序，按序执行 `get_for_update`
 - 4: 检查源对象与目标对象的类型兼容性
 - 5: **if** 目标存在且为非空目录 **then**
 - 6: 返回 `ENOTEMPTY`
 - 7: **end if**
 - 8: **if** 目标存在 **then**
 - 9: 按覆盖删除逻辑处理目标对象
 - 10: **end if**
 - 11: 删除源目录项，写入目标目录项
 - 12: 更新被移动对象的 `ctime`
 - 13: **if** 移动的是目录且跨父目录 **then**
 - 14: 对旧父目录追加 `IncrementNlink(-1)`
 - 15: 对新父目录追加 `IncrementNlink(+1)`
 - 16: **end if**
 - 17: 对涉及的父目录追加时间戳增量
 - 18: 提交事务并返回被覆盖对象列表
-

`rename` 的主体可以概括成“删旧边、插新边，必要时按覆盖删除处理目标”。边中心建模的好处在这里直接体现：即使跨目录移动，也只需改动少量与源路径、目标路径直接相关的记录，不必对整棵子树做级联改写。实际的复杂性来自 POSIX 语义约束而非底层搬移。

排序加锁对 `rename` 的意义不止于避免死锁。它让源目录、目标目录和被覆盖对象放在同一个一致事务视图下，使“检查目标是否存在”、“检查目标目录是否为空”、“删除旧目录项”、“写入新目录项”、“处理覆盖删除”这一组动作共享同一视图。在外部观察者看来，`rename` 表现为一次原子切换，不暴露“旧名字已删除、新名字尚未建立”这类中间状态。

父目录 `nlink` 只在目录跨父目录移动时才需要调整。普通文件移动不影响

任何目录的 `nlink`；同一父目录内的目录重命名也不影响父目录 `nlink`。这些差异直接决定了增量日志中需要追加哪些 `DeltaOp`——`rename` 的正确性不仅在于目录项改动对不对，还在于相关目录属性是否被同步维护。

3.5.4 readdir

`readdir` 是目录项键编码的直接受益者。在给定父目录 `inode` 之后，`MetadataServer` 构造前缀 `[D][parent: u64 BE]`，用 `RocksDB` 的前缀迭代器一次顺序扫描就能拿到该目录下全部子项。大端序编码保证同一父目录的所有条目在键空间中连续，前缀 `Bloom` 过滤器也在这一基础上生效。目录项 `value` 中已保存了子对象的 `mode`，`readdir` 在遍历时就能判断文件类型，不必为每个条目再回查一次 `inodes` 列族；这对目录下有大量条目的场景直接减少了读放大。

`readdirplus` 在此之上再做一层：把每个子项的完整属性也一起返回。这要用到折叠后的 `inode` 视图——对每个子 `inode` 都要依次查 `InodeFoldedCache`、未命中则扫 `delta_entries` 折叠、再写回缓存。这部分开销在冷启动时较明显，热态下基本由缓存吸收。

3.6 本章小结

本章围绕 `inode` 与目录项两个核心对象，集中讨论了 `MetadataServer` 内部的元数据组织与事务模型。目录树通过“以边为中心”的大端序键编码映射到 `RocksDB` 键空间，使 `create`、`unlink`、`rename` 等操作对应对一条目录边的增删；元数据按访问模式划分到 `inodes`、`dir_entries`、`delta_entries`、`system` 四个列族，独立配置压缩和过滤器；父目录属性的高频更新通过 `DeltaOp` 追加 + 懒折叠机制绕开单条基值键的热点，配合 `InodeFoldedCache` 控制读路径折叠成本；并发正确性则由 `RocksDB` 悲观事务兜底，`get_for_update` 把检查与修改收拢在同一事务视图内，排序加锁避免跨对象死锁，锁等待超时后由服务端有限重试消化偶发冲突。这套键编码、事务与 `DeltaOp` 的组合是本章的核心设计；一次完整的用户请求还需要 `FUSE` 客户端、`MetadataServer` 和 `DataServer` 三段协同，下一章从系统实现的角度把这三段串起来。

4 系统实现

第3章集中讨论了 MetadataServer 内部的元数据组织与事务模型。一次完整的用户请求还需要 FUSE 客户端、MetadataServer 与 DataServer 三段协同才能完成，本章从系统实现角度把这三段串起来。内容涵盖代码组织与语言选择、客户端到两个服务的请求分发与编排、MetadataServer 与 DataServer 各自的实现、跨服务的 gRPC 协议与连接层优化，以及正常路径之外的错误处理与恢复边界。

4.1 实现概览

RucksFS 是一个 Rust workspace，由 7 个 crate 组成。rucksfs-core 定义三组抽象——MetadataOps、DataOps、VfsOps——作为整条路径上各段的接口契约；rucksfs-storage 实现 RocksDB 和原始磁盘两类后端；rucksfs-server 和 rucksfs-dataserver 分别是 MetadataServer 与 DataServer 的进程实现；rucksfs-client 提供 FUSE 客户端与请求分发层；rucksfs-rpc 用 tonic 实现 gRPC 通信层；最外层的 rucksfs crate 负责把上述组件装配成可直接运行的二进制文件。按抽象优先而非具体优先的拆分方式，使同一套元数据逻辑既能以嵌入式方式在单进程内运行（All-in-One 模式），也能部署成分布式三段式服务。

图 4-1 给出系统整体架构与各组件的内部分层。

从左到右分别是 FUSE 客户端、MetadataServer、DataServer 三个节点。客户端内部自上而下依次是 FUSE 协议层、请求分发层和两个 gRPC 存根；元数据链路从请求分发层经 gRPC 进入 MetadataServer，数据链路单独经 gRPC 进入 DataServer。MetadataServer 内部聚合了事务层（目录锁与 rename 统一锁序）、缓存层（InodeFoldedCache）、增量引擎（DeltaOp 懒折叠）与后台折叠线程，所有状态落在 RocksDB 的四个列族里；DataServer 内部只有 DataService 接口和 RawDiskStore 两层，数据直接落到本地磁盘。图中两条 RPC 链路对应本章后续讨论的两条独立路径，客户端是唯一能同时访问两个服务的角色。

选择 Rust 的原因有二。一是文件系统对内存安全要求较高，生命周期与所有权模型使得并发共享状态的管理可以在编译期完成检查，减小运行时出现越界或 use-after-free 的风险。二是 tokio 异步运行时与 FUSE 的异步回调模型契合

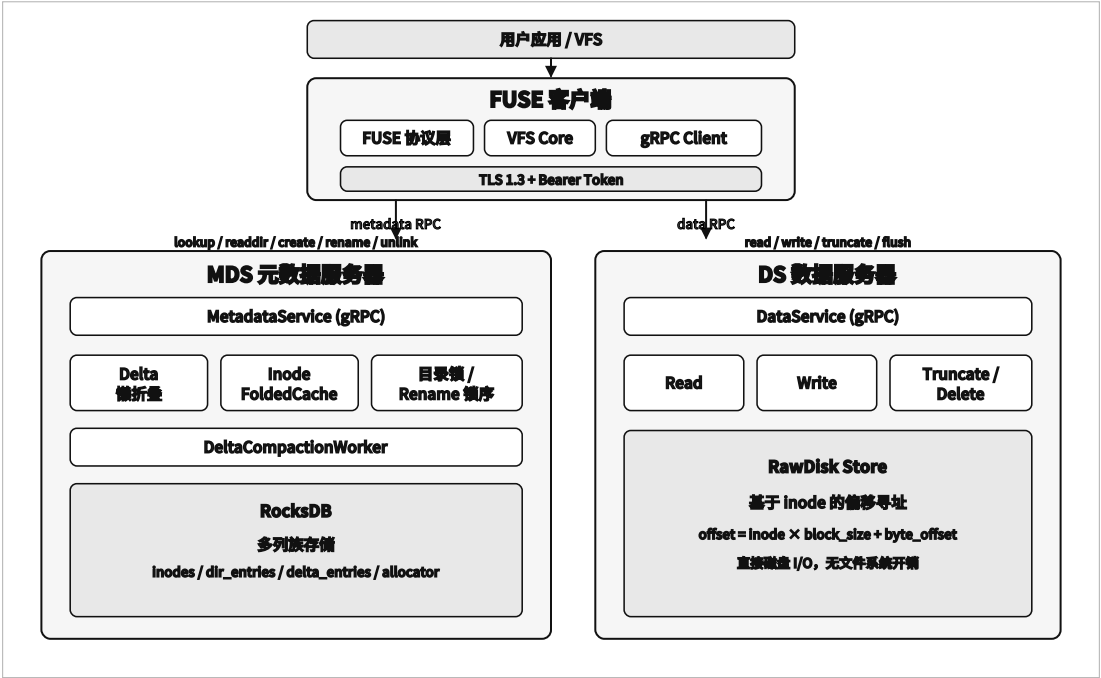


图 4-1 RucksFS 系统整体架构

——FUSE 的典型请求处理需要跨越 FUSE、gRPC、RocksDB 三层 I/O 边界，协程加阻塞等待的写法在这种多阶段 I/O 场景下既能维持较高并发度，又比”一请求一线程”的模型更省资源。本文涉及的 gRPC 层基于 tonic 0.12 实现，FUSE 层基于 fuse3 0.8。

后续各节按”客户端 → MetadataServer → DataServer → RPC 与连接层 → 错误处理”的顺序展开。

4.2 客户端：从 FUSE 到两个服务

客户端由三部分组成：FUSE 协议层直接面向内核，请求分发层把每类调用映射到对应的后端，远端存根负责与 MetadataServer 和 DataServer 的 gRPC 通信。本节先讲 FUSE 层的工作方式和挂载配置，再讲请求如何在两个服务之间分发，最后以 open 为例说明典型的跨服务编排流程。

4.2.1 FUSE 层与挂载配置

FUSE 客户端使用 fuse3 0.8 库实现，启用 tokio-runtime 与 unprivileged 两个特性。前者让 FUSE 回调直接运行在 tokio 异步运行时之上，当一次回调需要等待后端 RPC 返回时可以让出协程、让执行器推进其他请求；后者则允许以普通用户身份挂载 FUSE，不依赖 setuid 的 fusermount 帮助程序，便于在受限

容器环境里部署。

挂载阶段的两项关键配置是 `default_permissions` 和 `allow_other`。前者让内核 VFS 在请求进入 FUSE 客户端前完成标准的 `owner/group/other` 权限检查——客户端拿到的请求都已经过内核过滤，不需要重复实现 POSIX 权限语义；后者让挂载点对其他 `uid` 可见，便于多用户或多进程访问。目录项与属性缓存的 TTL 由客户端根据场景配置：生产部署下 TTL 通常设为秒级以吸收重复查询；第 5 章的性能测试把 TTL 设为 0 以测量 MDS 纯路径的吞吐。

4.2.2 请求分发与缓存

请求分发层按请求与后端的依赖关系把一次 FUSE 调用分成三类。

第一类是纯元数据操作，包括 `lookup`、`create`、`mkdir`、`unlink`、`rmdir`、`rename`、`readdir`、`getattr`、`setattr`。这些调用只涉及命名空间和 `inode` 状态，直接转发给 `MetadataServer` 即可。

第二类是纯数据操作，包括 `read`、`write`、`truncate`、`fsync`。这些调用需要知道目标 `inode` 的数据落在哪一台 `DataServer` 上，但这类路由信息在 `open` 阶段已经获取并缓存在文件句柄里，客户端不必每次 I/O 都问 `MetadataServer`，直接按句柄中记录的服务器编号路由即可。

第三类是跨服务的复合操作，其特点是一次调用需要先后触及两个服务。例如 `open` 要先从 `MetadataServer` 拿到 `DataLocation` 才能用于后续 I/O；`release` 对应 `close`，在 `unlink` 后句柄归零时还需要到 `DataServer` 侧做数据清理；覆盖式 `rename` 和带截断的 `setattr` 则是在元数据事务返回后，客户端再根据响应结构（`purged_inodes`、`needs_truncate` 等字段）补一个数据侧的清理或截断请求。

这样划分之后，命名空间与 `inode` 状态的语义约束全部留在 `MetadataServer` 内、字节读写与清理留在 `DataServer` 内，跨两者的复合语义由客户端显式编排。边界清楚的直接好处是出问题时更容易定位。代价是客户端比本地文件系统中只做请求转发的客户端更复杂——这是分离式架构必然要承担的一层成本。

4.2.3 以 open 为例的跨服务编排

三类操作中第三类最能体现跨服务编排。以打开一个已存在的文件 `/a/b` 为例，完整流程见图 4-2。

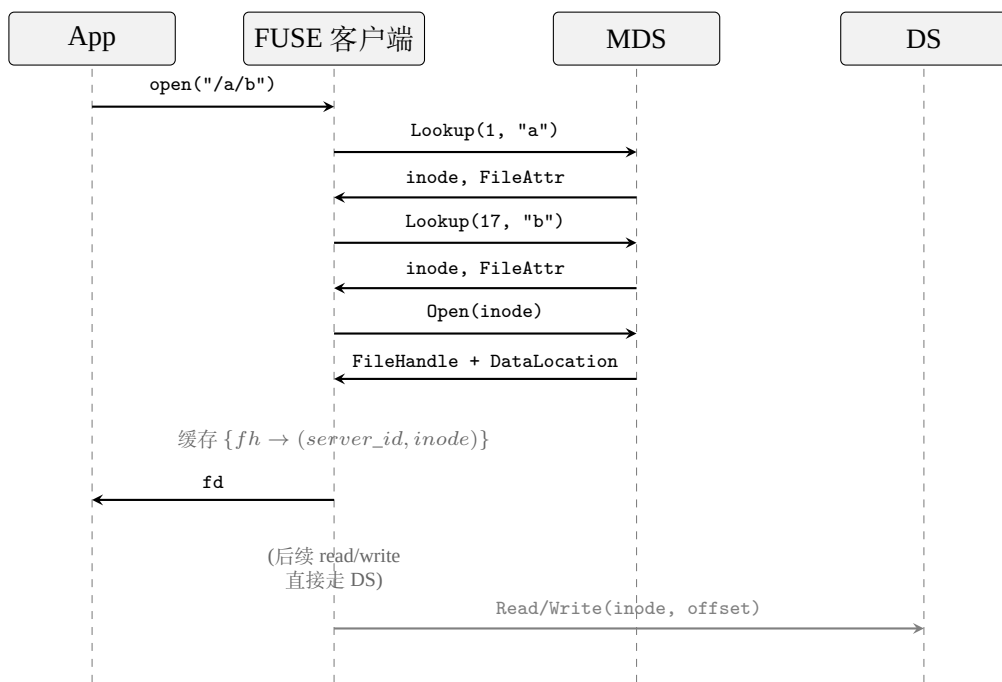


图 4-2 open 跨服务编排的请求时序

应用发起 `open("/a/b")` 后，FUSE 客户端需要先把路径解析到目标 inode——向 MetadataServer 发两次 Lookup，第一次取 a、第二次取 b，每次都拿回 inode 和完整属性。拿到目标 inode 后再发一次 Open，服务端为此 inode 创建一个文件句柄，并连同对应的 DataLocation（其中包含 DataServer 编号和 inode）一起返回。客户端在本地维护句柄表 $\{fh \rightarrow (server_id, inode)\}$ ，后续对该句柄的 read/write 直接按 server_id 路由到 DataServer，不再需要经过 MetadataServer。这样数据 I/O 的稳态路径只涉及一次 gRPC 往返，open 阶段一次性付清的路由代价换来了后续每次 I/O 的快路径。

路径深度每增加一层，就会多一次 Lookup 往返。这是 FUSE 客户端在关闭属性缓存时每次 create 需要多走两次 lookup RPC 的原因；是否放宽 TTL 以消化这部分开销是一个部署层面的取舍。

4.3 MetadataServer 实现

MetadataServer 的职责在第 3 章已经讨论过——维护命名空间与 inode 状态，处理 DeltaOp 折叠，保证并发目录操作正确。本节从工程实现角度补充三件事：进程内各模块如何分工、异步请求与阻塞事务如何共存、启动时如何完成持久化状态的初始化。

4.3.1 模块分工

`rucksfs-server` crate 按职责切分成几个文件。`lib.rs` 是主流程，实现 `MetadataOps` 的所有 POSIX 接口；`delta.rs` 定义 `DeltaOp` 编码并封装读路径折叠；`compaction.rs` 是后台折叠线程（`DeltaCompactionWorker`），周期扫描 dirty inode 列表，超过阈值时把增量折叠回基值；`cache.rs` 实现 `InodeFoldedCache`，16 分片 LRU，默认容量 10 000；`fsck.rs` 提供启动期的一致性检查，用于离线排查问题。持久化层由 `rucksfs-storage` 提供：`rocks.rs` 实现 RocksDB 后端的 `MetadataStore`、`DirectoryIndex`、`DeltaStore` 与 `AtomicWriteBatch`，`encoding.rs` 定义 `InodeValue` 的定长序列化与所有键的字节布局，`allocator.rs` 负责 inode 编号分配。

这一拆分让“事务语义与键编码”两件事分别落在独立 crate 里——`rucksfs-server` 不直接调用 RocksDB 接口，而是通过 `rucksfs-storage` 暴露的 trait 访问后端。切换后端时上层不用改，离线测试时用内存实现替换后端也比较容易。

4.3.2 异步请求与阻塞事务的搭配

RocksDB 的 Rust 绑定是同步阻塞的。但 RucksFS 的对外接口（gRPC 服务与 FUSE 回调）都在 `tokio` 异步运行时上。两者结合的做法是：gRPC 请求处理器以 `async fn` 形式定义，但实际落到 RocksDB 的事务提交时，先用 `tokio::task::spawn_blocking` 把阻塞调用切到阻塞线程池执行，提交完成后再把结果通过 `future` 交还给原请求。这样异步请求池不会被阻塞调用阻塞住，且事务本身依然运行在专门用于阻塞 I/O 的线程里，CPU 调度上不冲突。

另一处需要注意的是共享状态。MetadataServer 的 RocksDB 句柄、

InodeFoldedCache、inode 分配器这些内部状态都用 Arc 共享到请求处理器和后台线程里。缓存分片内部持有 `parking_lot::Mutex`，相比标准库的 `std::sync::Mutex` 更轻量，在高并发 `getattr` 场景下锁竞争面积较小。

4.3.3 启动与持久化

MetadataServer 启动时做几件事：初始化 RocksDB 实例，按设计打开 `inodes`、`dir_entries`、`delta_entries`、`system` 四个列族；检查根 inode（编号固定为 1）是否已存在，不存在则写入——这使得每次挂载都能从一致的根目录开始工作；启动 `DeltaCompactionWorker` 后台线程；最后绑定 gRPC 服务端口对外提供服务。根 inode 编号固定为 1 这一选择带来的副作用是：重启后客户端看到的路径解析入口总是同一个 inode，与上次运行时一致，不会因 inode 分配器内部状态漂移而导致根目录换人。

持久化保证由 RocksDB WAL 提供。所有写事务在提交时会先写 WAL，崩溃后重启时依据 WAL 回放未落盘的 MemTable。尚未提交的事务不会被当成“已经成功”——只要事务返回成功给客户端，对应的修改在磁盘上就一定可恢复。

4.4 DataServer 实现

DataServer 的实现刻意简化，目的是让元数据路径的性能差异不被复杂的块管理细节干扰。本节分别讲它的服务层和存储后端。

4.4.1 服务层

DataServer 是 `rucksfs-dataserver crate` 的全部内容，结构非常薄——对 `rucksfs-storage` 的 `DataStore trait` 做一层服务化封装，把字节读写请求以 gRPC 形式暴露出去。接口包括 `Read`、`Write`、`Truncate`、`DeleteData`、`Flush` 五个调用，全部直接对应底层 `DataStore` 方法；服务端本身不维护复杂状态，不涉及命名空间，也不参与事务。这层封装的作用主要是为将来引入多 DataServer 或换用其它存储后端留出抽象边界。

4.4.2 RawDiskDataStore：固定偏移映射

当前唯一的 `DataStore` 实现是 `RawDiskDataStore`。所有 inode 的内容共享同一个文件 `data.raw`，每个 inode 分配一段固定大小的逻辑地址区间，读写偏

移按公式

$$\text{disk_offset} = \text{inode} \times \text{max_file_size} + \text{file_offset}$$

计算，再用 `pread/pwrite` 执行。`max_file_size` 在启动时传入，默认 64 MiB。DataServer 内部不维护空闲空间位图、extent 树或块分配器——inode 编号本身就是数据文件上的一个逻辑槽位。

这种设计的好处是直接的：数据路径不需要块分配逻辑，寻址是常数时间，服务端状态极简。对本文关注的元数据优化而言还有一项附加好处：数据侧越简单，第 5 章实验中观察到的性能差异就越容易归到元数据组织、客户端协调和网络通信上，不会被数据块管理细节干扰。

代价也直接。每个 inode 预留 `max_file_size` 大小的逻辑地址空间——默认 64 MiB——在大量小文件场景下空间利用率偏低。`delete_data` 目前为空操作，对象删除后物理空间不会立即回收。稀疏文件能缓解部分空间浪费（实际占用以写入的块数计），但稀疏不是真正的空间管理。这些短板是当前原型有意接受的，为了让论文聚焦在元数据路径上；真实生产场景下需要补上正式的块分配器、空间回收与持久化对象删除。

并发语义上，写入偏移由调用方指定，不同客户端向同一 inode 的不同偏移写入不会因 DataServer 内部地址分配而产生额外竞争；`pread/pwrite` 以调用为边界完成单次 I/O。若两个写请求本身覆盖了重叠区域，属于上层应用的并发语义问题，不在 DataServer 的责任范围内。

4.5 RPC 与连接层

gRPC 在这条路径上不只是通信。接口长什么样、一次 RPC 携带多少信息、连接层怎么组织，都会影响客户端与服务端之间的往返次数，进而影响整体性能。本节按接口设计原则、接口分层、连接池优化三部分讨论。

4.5.1 接口设计原则

RucksFS 的 gRPC 接口遵循两条原则。第一条是元数据操作与数据操作严格分离：两套服务定义互不交叉，客户端对一个请求一眼就能看出应该走元数据链路还是数据链路。第二条是尽量让一次 RPC 返回下一阶段所需的全部信息，避免把简单操作拆成多轮往返。例如 `Open` 的响应里既有 `FileHandle` 又

有 `DataLocation`，客户端不必再发一次“问路由”的调用；`Unlink` 的响应里携带 `purged_inodes`，让客户端直接知道哪些 `inode` 需要到 `DataSeter` 侧清理；`SetAttr` 的响应里有 `needs_truncate` 指示客户端是否需要补一个截断请求。

4.5.2 接口分层

`MetadataServer` 的 RPC 接口按职责分成四类：命名空间操作（`Lookup`、`Create`、`Mkdir`、`Unlink`、`Rmdir`、`Rename`、`Readdir`）、属性操作（`GetAttr`、`SetAttr`）、句柄操作（`Open`、`Release`、`CreateAndOpen`）以及写入后的元数据协调（`ReportWrite`）。`DataSeter` 的接口则刻意收敛，只包含 `Read`、`Write`、`Truncate`、`DeleteData`、`Flush` 五个直接面向字节内容的调用。这样从接口层面就能判断“谁负责对象语义、谁负责数据字节”。

`CreateAndOpen` 是一个专门的合并接口，把新建文件的事务与句柄分配合并到一次 RPC 完成。典型的 `open(O_CREAT)` 在客户端层面会先 `Lookup` 判断路径是否存在、若不存在再 `Create` 插入记录、最后 `Open` 获取句柄。把创建与打开合并到一个请求里之后，当客户端明确希望“没有则新建、有则直接打开”时，只需一次 RPC，省掉中间两次往返。这是本章在减少 RPC 次数这一方向上已经做的一项具体优化。

4.5.3 多通道连接池

连接层的工程优化点比较集中。`tonic` 建立的每一个 `Channel` 对应一条独立的 HTTP/2 连接；HTTP/2 帧在单条 TCP 连接上是顺序序列化的，客户端发出的请求再多，单条连接上写帧这件事本身也只能按顺序完成。本地压测下这会直接成为瓶颈：单条连接的 `create` 吞吐上限在 34 000 ops/s 左右；继续加压，服务端有富余 CPU，但发不出更多请求。

解决办法是让客户端维护多条独立的 `Channel`，每次 RPC 以轮询方式选一条发送。池的大小由环境变量 `RUCKSFS_CLIENT_POOL_SIZE` 配置，默认为 1；第 5 章的高并发测试实际运行时把池大小设到 4 到 8 之间。多通道之间相互独立，帧序列化瓶颈在连接池维度被打散，服务端的多核处理能力才能用上。代价是客户端需要维护一组空闲连接，每条连接各自维持 `keepalive`——对客户端来说内存与 FD 开销略增，但在分布式文件系统的场景下这些开销可以忽略。

连接层还有一项细节——内部做过一次 keepalive 调优。tonic 的默认 HTTP/2 超时设置对短连接场景比较合理，但 FUSE 客户端通常一挂就是小时级别；默认配置下偶尔会有连接被服务端侧关闭、客户端下一个请求发现失败再重建的情况。当前实现显式启用了 HTTP/2 keepalive，周期小于空闲超时，避免连接被中间网络设备或对端无感知地关闭。

4.6 错误处理与恢复

从“单进程存储引擎”扩展到“客户端 + MetadataServer + DataServer”之后，需要处理的失败类型也变多了：事务冲突、RPC 失败、服务崩溃各自有不同的错误来源，不能一律按“重试”处理。本节从错误码映射和崩溃恢复两方面把当前实现的故障边界说清楚。

4.6.1 错误码映射与重试策略

第一类是 MetadataServer 内部的事务冲突。这是并发目录操作最常见的失败——多个事务争夺同一行锁，等待超时后 RocksDB 返回 TransactionConflict。本文没有把它直接抛给用户，而是在服务端先做有限重试：最多 3 次，起始退避 50 μ s、每次翻倍。只有重试仍然失败，才以 TransactionConflict 向上返回，经 gRPC 传到客户端后在 FUSE 层映射为 EAGAIN。这样偶发冲突不会转化为用户可见错误；EAGAIN 的语义也比较精确——只表达“事务冲突暂未消解”，不与任意 I/O 异常混在一起。

第二类是 RPC 或本地 I/O 失败。当前原型对这类失败的处理相对保守：客户端按 gRPC 返回的错误状态向上报对应的文件系统错误，不在数据路径上维护待确认的写队列，也没有做基于 inode + offset 的幂等重发。换言之，当前只实现了“事务冲突自动重试”这一项，跨服务失败会直接向应用暴露。真实生产部署下，数据路径的幂等重发、连接重建之后的状态恢复是必须补齐的；论文聚焦于元数据路径的验证，这部分工作留作后续。

4.6.2 服务崩溃的恢复边界

MetadataServer 崩溃时，RocksDB TransactionDB 的 WAL 提供崩溃一致性：已经提交但尚未刷入 SST 的内容在重启后由 WAL 回放恢复，尚未完成提交的事务不会被当作“已经成功”。因此只要某次元数据操作已经向客户端返回成功，

其持久化结果就不会因 `MetadataServer` 进程崩溃而丢失。客户端侧会把崩溃感知为一次 RPC 失败或重试；必要时 `open` 可以重新向 `MetadataServer` 请求新的 `DataLocation`。

`DataSeter` 的崩溃语义较简单，也受当前实现边界的约束。若负责某文件的 `DataSeter` 不可达，对应的 `read` 或 `write` 直接返回 `EIO`——当前原型没有数据副本也没有多 `DataSeter` 故障切换，`DataSeter` 不可达不会被透明屏蔽，而是直接暴露为“该文件暂时不可访问”。`delete_data` 为空操作，也就不存在“删除已提交但物理空间尚未回收”这一层恢复问题。

4.6.3 讨论范围

本文的讨论范围受若干明确边界约束。当前 `RucksFS` 保持“单 `MetadataServer` + 单默认 `DataSeter`”这条主路径。`DataLocation` 的字段设计和客户端的数据路由都为多 `DataSeter` 预留了接口，但元数据分片、多副本一致性、物理空间回收、严格的双阶段 `fsync` 都尚未实现。本文的研究重点是元数据面和数据面分离之后，如何通过键编码、事务控制、客户端协调和增量更新来优化元数据关键路径；第 5 章的实验也是在这条主路径上完成的，不声称系统已经覆盖多副本容错、跨节点故障转移或空间回收等更大范围的能力。

4.7 本章小结

本章从实现角度把第 3 章的元数据设计嵌入到一个完整的分布式原型里。代码按 7 个 `crate` 组织，上层抽象 `trait` 与具体后端实现分离；`Rust` 的异步运行时让 `FUSE`、`gRPC`、`RocksDB` 三层 I/O 在同一套协程模型下协同工作。客户端把请求按类别分发到 `MetadataServer` 或 `DataSeter`，对跨服务的复合语义（`open` 获取数据路由、`release` 触发数据清理等）在客户端一侧统一编排；`MetadataServer` 用阻塞线程池承接 `RocksDB` 事务，配合 `InodeFoldedCache` 和后台折叠线程，把设计章的元数据逻辑落到真实进程里；`DataSeter` 走最小实现的路径，用固定偏移映射换取数据侧的简洁与可解释性。`gRPC` 接口的设计目标是让一次调用尽量覆盖下一阶段所需的全部信息，`CreateAndOpen` 的合并和多通道连接池是这条思路在接口与连接两层的具体落点。错误处理按事务冲突、RPC/IO 失败、服务崩溃三层展开：事务冲突由服务端有限重试消化，RPC/IO 失败直接透明上抛，

MetadataServer 的 WAL 提供崩溃一致性, DataServer 不可达则暴露为 EIO。本文明确把多副本、元数据分片、空间回收和严格双阶段 `fsync` 划在讨论范围之外; 第 5 章的实验将在这一主干之上检验前两章的设计是否达到了预期。

5 测试与结果分析

第 3 章给出了 RucksFS 在元数据组织、事务模型、增量更新三条线上的设计，第 4 章说明了从 FUSE 客户端到 MetadataServer、DataServer 的跨服务协调方式。本章把这些设计放到真实分布式部署下，检验其是否达到预期。

5.1 研究目标与评估线索

本文的研究对象是元数据键值存储层面的文件操作，落到系统上即 RucksFS 的 MetadataServer (以下简称 MDS)。MDS 承担目录项编码、inode 属性维护、事务提交、DeltaOp 增量更新等元数据逻辑，本章所关心的是这条路径的语义正确性和吞吐表现。

围绕这一目标，实验分为两条独立的线。一条是 POSIX 语义合规性，采用 `pjdfstest` 套件，在分布式部署下系统性地验证 RucksFS 是否给出了与 POSIX 规则一致的行为。另一条是元数据性能，采用 `mdtest`，在同一套硬件上对 RucksFS、NFS v4.2 和 JuiceFS+TiKV 三套系统作横向对比，并对 RucksFS 自身的 Delta 与 NoDelta 两种构建作内部对照。

两条线的目标不同，本章能够和不能够支撑的结论也不同。正确性这一线能够支撑的结论是：在实现范围内，RucksFS 在分布式部署条件下给出了与 POSIX 规则一致的响应。性能这一线能够支撑的结论是：在 MDS 自身负责处理每一次元数据请求的前提下，RucksFS 的元数据路径相对上述两条常见工程路线具有怎样的吞吐与扩展性。本章不声称，也不试图覆盖，包含客户端缓存在内的应用视角完整性能比较——那需要把三套系统在生产默认配置下再测一轮，属于后续工作。

5.2 POSIX 合规性评估

5.2.1 测试工具与测试环境

`pjdfstest`^[38] 是业界常用的 POSIX 文件系统合规性测试套件，覆盖 `chmod`、`chown`、`link`、`mkdir`、`mkfifo`、`mknod`、`open`、`rename`、`rmdir`、`symlink`、`truncate`、`unlink`、`ftruncate` 等 13 类系统调用。套件中每条 POSIX 规则都会按文件类型

(regular、dir、symlink、fifo、socket、char、block) 展开成多条用例，目的是系统地验证同一条规则在各种对象上的一致性。本文采用 pjdfstest 的 Rust 实现版本，总共 398 条用例。

这种展开方式对被测系统较为严苛：同一条 POSIX 规则在各种文件类型上共用同一代码分支，只要其中任一类型不被支持，整组用例都会记为失败，即便被测系统对该规则的核心语义本身是正确的。因此直接使用总通过率衡量语义完整度并不合适，需要先划定「实现范围内」的用例集，再讨论结果。

本节测试在分布式部署下进行：FUSE 客户端与 MetadataServer 位于不同的物理节点，两者通过 gRPC 通信；DataServer 与 MetadataServer 同机部署。这一部署要求一条用例能够通过，不仅要 MetadataServer 内部的事务逻辑正确，还要错误码在穿过 gRPC 和 FUSE 两层之后依然保持不变。

5.2.2 实现范围与理论排除项

在给出测试数据之前，先根据第 3 章的设计边界把 398 条用例划分到两类中。

排除项 A：mknod 与 mkfifo 相关用例。这两个接口管理的是 Unix 域 socket、命名管道、字符设备节点和块设备节点，运行语义与内核设备号、内核管道缓冲区紧密绑定，与本文「通过键值对持久化命名空间」的研究主题正交。结合本文关注的典型负载（容器镜像、训练数据、日志、构建产物），这两个接口对论证元数据机制本身帮助不大。因此本文明确选择不实现它们，RucksFS 对二者统一返回 EOPNOTSUPP，即 POSIX 规定的「操作不支持」错误码，用户程序会明确得到「文件系统不支持该操作」的结果，而不是被伪装为成功。凡是直接调用 mknod/mkfifo 的用例，以及需要先用它们准备测试对象的派生用例，均属于此类。以 pjdfstest 中「当 rename 的目标路径前缀不是目录时应返回 ENOTDIR」一条规则为例，该规则会按 7 种文件类型各生成一条用例，其中后 4 种需要先用 mknod/mkfifo 准备 fifo、socket 或设备节点；用例还未进入 rename 的语义检查即因 EOPNOTSUPP 而中断——而 RucksFS 在常规文件、目录、符号链接上对同一条规则都给出了符合规则的响应。

排除项 B：DataServer 单文件大小上限。作为最小实现，当前

RawDiskDataStore 使用固定偏移公式映射 inode，单文件逻辑上限为 64 MiB。open::interact_2gb 这条用例直接位于该上限之上，属于 DataServer 的开发期参数，与元数据设计所要回答的问题无关。

排除项 C：与元数据设计无关的跳过项。 pjdftest 框架本身会跳过 48 条用例：27 条来自 utimensat 系列（需要单独的 feature flag，当前未启用），21 条涉及 remount（FUSE 配置不允许运行时重新挂载）和跨文件系统操作（测试环境未配置第二个文件系统）。这些项与 RucksFS 的元数据设计无关。

扣除上述三类理论排除项之后，剩下的即为「本文实现范围内应当通过」的用例集。表 5-1 给出分类汇总。

表 5-1 pjdftest 用例按本文实现范围的划分

类别	用例数	说明
实现范围内	182	本文选择实现的接口，应当全部通过
排除项 A（mknod/mkfifo 相关）	167	直接或前置依赖这两个接口的用例
排除项 B（单文件大小上限）	1	open::interact_2gb
排除项 C（框架跳过）	48	utimensat、remount、跨文件系统
合计	398	

5.2.3 实现范围内的测试结果

在上述划定的 182 条「实现范围内」用例上，RucksFS 全部通过，通过率 100%。换言之，只要一条 POSIX 规则不涉及本文明确不实现的接口，RucksFS 在分布式部署条件下都给出了符合语义的响应。这一结果是本章后续性能数据的语义前提。

为了让结论落到具体数据上，表 5-2 按 13 类系统调用给出完整分布。表中 rename、unlink、link 等类别的失败数看起来较高，但失败全部集中在前文所述「需要 mknod/mkfifo 准备测试对象」的派生用例上——每条规则展开的 7 种文件类型中，后 4 种在前置阶段即告中断。若按派生用例实际走到的代码路径折算，这几类在「实现范围内」同样是全部通过的。

5.2.4 与第 3 章设计决策的对照

仅凭「实现范围内 182 条全部通过」这一汇总数据，尚不足以说明 RucksFS 确实把第 3 章的若干设计决策落到了实处。为避免「因为未实现所以不会失败」一类误读，下面挑选几条较易被忽视的边界语义加以说明。这些用例都位于实

表 5-2 pjdftest 各操作类别的结果分布

操作	通过	失败	跳过	合计	通过率
ftruncate	6	0	0	6	100.0%
truncate	14	4	1	19	73.7%
open	18	7	2	27	66.7%
chown	16	8	2	26	61.5%
rmdir	14	8	1	23	60.9%
mkdir	12	8	1	21	57.1%
symlink	12	11	1	24	50.0%
chmod	16	16	1	33	48.5%
mkfifo	9	11	1	21	42.9%
mknod	15	23	0	38	39.5%
rename	23	28	9	60	38.3%
unlink	13	20	1	34	38.2%
link	14	24	3	41	34.1%

现范围内、都在通过列表之中，且每一条都可以直接对应到第 3 章的某个具体设计决策。

- **unlink::open_file_not_freed** (对应：删除语义与句柄生命周期，第 3.4 节)。文件被 unlink 之后，若仍有打开句柄，应当能够继续读写，物理回收需推迟到最后一个句柄关闭。这要求 MetadataServer 同时维护 nlink 和打开计数：当 nlink 归零但计数仍大于零时，对象进入 pending_deletes，由 release 触发最终清理。若事务未将「删除目录项」和「维持 inode 可达性」分开处理，该用例将不通过。
- **rename::to_multiply_linked::regular** (对应：rename 覆盖逻辑，算法 3-2)。rename 的目标若是带有多条硬链接的文件，覆盖动作只能将目标的 nlink 减一，而不能直接删除目标 inode。这点对应第 3 章 rename 流程中「按覆盖删除逻辑处理目标对象」一步。
- **12 条 *::enametoolong_component 用例** (对应：错误码穿透)。文件名长度超过 NAME_MAX 时应返回 ENAMETOOLONG。该约束在 MetadataServer 所有接受 name 字段的 RPC 入口统一校验，错误码需经 gRPC 和 FUSE 两层传递后保持不变。
- **open::open_trunc** (对应：客户端跨服务编排，第 4.2 节)。以 O_TRUNC 打开已有文件时，size 需在打开操作中同步清零，同时刷新 mtime 和 ctime。FUSE 不一定会再发一次独立的 setattr，因此这些工作必须在 open 实现

中一并完成。

- `rmdir::eexist_enotempty_non_empty_dir` 系列（对应：事务内空目录检查，第 3.5 节）。删除非空目录应返回 `ENOTEMPTY`。第 3 章已说明，`rmdir` 把空目录检查也纳入事务之内，在一致性视图下执行前缀扫描，避免检查结果在进入事务前失效。

这些用例覆盖的并非孤立的接口行为，而是第 3 章的跨列族原子提交、句柄生命周期、统一加锁顺序以及客户端协调路径在实际运行时的综合表现。它们能够全部通过，说明这套设计在分布式部署条件下达到了预期的语义效果。

5.2.5 小结

综上：在明确排除 `mknod/mkfifo` 相关用例、`DataServer` 单文件大小用例以及 `pjdfstest` 框架自身的跳过项之后，剩下 182 条「实现范围内」用例全部通过，通过率 100%。排除项中的 167 条失败由 `pjdfstest` 按文件类型展开的机制所放大，并不构成对元数据正确性的实际反例；`mknod/mkfifo` 的不实现是面向典型分布式存储负载所作的显式取舍，在返回值上被标记为 `EOPNOTSUPP`，而非伪装为通过。结合前述几条边界用例的对照，`RucksFS` 的元数据实现在本课题要求的范围内达到了预期的语义正确性目标。这一结果同时也是后续性能数据的前提——下文的吞吐数据都建立在「实现范围内语义正确」的基础之上。

5.3 元数据性能评估

5.3.1 对比对象的选择

性能这一线主要回答两个问题：其一，`RucksFS` 基于 `RocksDB` 实现的元数据方案在性能上能否达到业界常见 `KV` 文件系统的水平；其二，相对传统本地文件系统，`KV` 这条路线是否带来了改善。围绕这两个问题，本节设置三组对比。

`RucksFS` 与 `JuiceFS+TiKV` 的横向对比针对第一个问题。两者都采用「`FUSE` 客户端加 `KV` 后端」这一结构，区别在于 `RucksFS` 的 `MDS` 直接操作本机的 `RocksDB`，针对元数据访问模式做过定制，而 `JuiceFS+TiKV` 把元数据放在一个通用分布式事务 `KV` 之上。这一对比衡量的是「为元数据访问模式做过定制的专用 `KV` 后端」与「通用分布式 `KV` 后端」两条工程路线之间的差距。

`RucksFS` 与 `NFS v4.2` 的横向对比针对第二个问题。`NFS v4.2` 服务端底层是

ext4 的目录 B+ 树，与 RucksFS 的 KV 方案在元数据存储模型上恰好对立。这里有一个取舍需要交代：本课题原本要衡量的是「KV 模型」与「传统目录 B+ 树模型」之间的差距，理论上直接与本机 ext4 对比最为干净，但 RucksFS 的 MDS 是「FUSE 客户端加网络 RPC 加远端服务」的分布式形态，本机 ext4 则完全在内核内完成，既没有 FUSE 的用户-内核切换，也不经过网络。与其直接对比，实测差距会被这两层额外变量整体抬高，难以把结果归到「存储模型」这一项上。NFS v4.2 的整体形态与 RucksFS 相近——都是客户端走网络协议到一个远端服务——但其服务端底层正是 ext4 的目录 htree。上层拓扑对齐之后，两侧的主要差异便集中在「元数据用什么数据结构组织」上，对比结果才能真正落到本课题关心的问题上。第 2 章讨论过 FUSE 自身的开销在这类分布式原型中并不占主导——真正决定整体延迟的是跨节点 RPC 往返——所以这一对齐方式在数量级上是合理的。

Delta 与 NoDelta 的内部对照针对 DeltaOp 机制本身的作用。两者的区别只有一个编译期特性开关：batch_parent_deltas 打开时，父目录属性更新走 DeltaOp 追加路径；关闭时退化为对父 inode 基值的读-改-写。其余代码完全一致。这样，二者之间的吞吐差距只能来自父目录更新路径的不同，与其他可能的干扰因素无关。NoDelta 仅用于对照，并非生产构建。

三组对比共享同一套硬件、同一份负载、同一次清场流程，可以相互引用数据。 $N \leq 32$ 区间内三套外部系统都能给出稳定的数据，本节据此完成横向对比； $N \geq 64$ 时 JuiceFS+TiKV 与 NFS v4.2 各自面临不同层面的限制（§5.3.5 节会具体说明），横向对比失效，因此高并发区间转用 Delta 与 NoDelta 的内部对照。

5.3.2 测量对象与缓存配置

本节性能实验的测量对象是 MDS 的处理能力——一次元数据操作从 FUSE 请求进入到服务端事务提交再返回这条路径上的真实延迟和吞吐。基于这一测量对象，对客户端缓存和服务端缓存需要分别处理。

客户端侧，内核 VFS 在收到 FUSE 返回的属性 TTL 之后会维护一份属性缓存，命中时 stat 请求不经过 FUSE 守护进程，也不到达 MDS，对 MDS 处理能力的测量没有贡献。NFS 客户端的属性缓存以及 JuiceFS 客户端的 attr/entry/dir-

entry 缓存与此性质相同。因此本节将三套被测系统的客户端缓存全部关闭：RucksFS 把 FUSE 返回的 entry/attr TTL 设为 0，JuiceFS 使用 `--attr-cache=0 --entry-cache=0 --dir-entry-cache=0`，NFS 以 `noac` 选项挂载。三套系统做法不同，但目的一致，都是让每次元数据操作都到达服务端。

服务端侧的缓存（MDS 自己维护的 `InodeFoldedCache`、`RocksDB BlockCache`、`TiKV` 的 `BlockCache`）是元数据引擎设计的组成部分，本节保留默认配置，由测试正常触发其命中行为。

NFS 在关闭客户端缓存这件事上还有一层额外的理由，需要一并说明。本节另外用默认 AC（Attribute Caching）配置在 $N \in \{2, 8\}$ 档位下尝试运行 `mdtest` 作为对照。 $N = 2$ 时三次 iteration 的 hard 模式 `create` 吞吐分别为 1742、45、1164 ops/s，相对 Mean 的标准差达 83%，其中第二次 iteration 几乎停滞。 $N = 8$ 时 `mdtest` 在第一次 iteration 中即因 `open64` 返回 `ENOENT` 触发 `MPI_ABORT` 而终止，错误路径为 `test-dir.0-0/mdtest_tree.0/file.mdtest.3.0`，该文件由对应 rank 自行创建，但客户端本地 `dentry` 缓存未及时刷新，被误判为不存在。NFSv4.2 默认 AC 配置下，目录属性与目录项缓存的生存期由 `acdirmin/acdirmax` 决定（30–60 秒），多客户端并发写入同一目录时，协议本身不提供足以支撑这种访问模式的一致性保证。这一现象是 NFS 共享父目录场景下的固有限制。

5.3.3 测试环境

集群拓扑。 测试集群部署在腾讯云香港二区（`ap-hongkong-2`），所有节点在同一 VPC、同一可用区内，内网 RTT 不超过 0.5 ms。集群由一台大规格服务器加若干等规格客户端组成，硬件配置见表 5-3。服务器选 64 核大规格，目的是让各档位下被测系统的元数据后端都不会在测试本身之前先达到 CPU 上限。客户端选 2 vCPU、2 GiB 的最小规格——在关闭 FUSE 属性缓存、每次 `create` 都穿透到服务端的条件下，单 rank 在该规格客户端上的 `create` 吞吐上限约为 400 ops/s，2 核即足以用满该上限；提高客户端规格不会改变结果，只会占用更多配额、降低可扩展的客户端数。所有客户端硬件规格保持一致，以排除横向不可比因素。

被测系统。 本节共设置四套被测系统，每次只运行一套，切换时在服务端

表 5-3 测试集群硬件配置

角色	机型	关键参数
Server	SA5.16XLARGE256	64 vCPU、256 GiB DDR5、200 GB CLOUD_BSSD
Client	SA5.MEDIUM2	2 vCPU、2 GiB DDR5、50 GB CLOUD_BSSD

客户端数量按测试档位递增，依次为 $N = 2, 8, 32, 64, 96$

操作系统：Ubuntu 22.04 LTS (kernel 5.15)

和客户端都执行完整的清场流程：停止守护进程、删除数据目录、确认监听端口释放、卸载挂载点并重建。整个流程由一个 orchestrator 脚本驱动，确保每次测试都从干净状态开始。四套系统的关键参数如下。

- **RucksFS-Delta**：默认构建，启用第 3.4 节描述的 DeltaOp 增量追加机制。父目录的 mtime、ctime、nlink 更新走增量路径。
- **RucksFS-NoDelta**：编译期通过 cargo 关闭 DeltaOp 特性，父目录属性更新退回对父 inode 基值的读-改-写，其余代码与 Delta 完全相同。
- **NFS**：Linux 内核 NFS v4.2 服务端 (nfs-kernel-server)，导出本地 ext4 目录，客户端以 vers=4.2,noac 挂载，nfsd 线程数为 128。
- **JuiceFS+TiKV**：JuiceFS 1.3.1，元数据后端为单节点 TiKV 8.5.6 (含 PD)，数据目录指向同机本地目录。该组合对应业界常见的「通用分布式 KV + 分层文件系统」路线。JuiceFS 与 RucksFS 共用 RocksDB 作为底层存储引擎 (TiKV 内部同样使用 RocksDB)，便于隔离上层架构差异。

评估工具与并发度设计。性能测试使用 mdtest 4.1.0^[39]，该工具是 IOR 套件的子项，也是 IO500 列表的元数据基准；跨客户端调度通过 OpenMPI 4.1.2 完成。每台客户端挂载一次目标文件系统，并通过 hostfile 机制在挂载点上启动一个或多个 mdtest rank。本节将并发度记为 $np = N \times r$ ，其中 N 是客户端台数， r 是每台客户端上 mdtest rank 的数量， np 即同时对服务端发起请求的 rank 总数。

具体档位的设计需结合一个前提：客户端缓存关闭之后，单 rank 在 2 核客户端上的 create 吞吐上限约为 400 ops/s。该上限来自 RPC 次数——一次 create 需要一次 lookup、一次 create、一次 release 共三次 RPC，RTT 约 700 μ s，加

上 FUSE 调度开销后接近该数值。横向对比这一线的 $N \in \{2, 8, 32\}$ 三档继续采用 $r = 1$ ，此时单 rank 尚未触及该上限，比较的是各被测系统在相同请求速率下的吞吐。内部对照这一线希望观察到服务端的事务冲突阈值，需要把到达 MDS 的请求速率再往上推；继续扩大 N 的话，每台客户端仍被 400 ops/s 所限，整体增速有限。因此 $N \in \{64, 96\}$ 两档下额外采用 $r = 2$ 与 $r = 4$ 两组设置，在客户端数不变的情况下把 np 继续向上推进。 $r = 4$ 时每台 2 核客户端运行 4 个 mdtest rank，理论上存在两倍的 CPU 超分配，但 mdtest 绝大多数时间阻塞在 RPC 返回上，CPU 并非瓶颈；实测 $r = 4$ 下 per-client 吞吐为 $r = 1$ 的 1.8–1.9 倍，说明客户端侧仍有余量。完整的 N 与 r 组合见表 5-4。

表 5-4 性能测试档位矩阵

用途	被测系统	N	r	np
横向对比	四套系统均参与	2, 8, 32	1	2, 8, 32
RucksFS 扩展性	Delta 与 NoDelta	64, 96	1	64, 96
DeltaOp 内部对比	Delta 与 NoDelta	64	2, 4	128, 256
		96	2, 4	192, 384
NFS 缓存配置对照	NFS（默认 AC 配置）	2, 8	1	2, 8

每 rank 固定创建 2 000 个文件，用于使单次 create/stat/remove 阶段有足够的采样时间（最大档位 np = 384 时总文件数达 768 000）。mdtest 的核心参数是 -F -C -T -r -i 3：-F 仅测试文件操作，-C/-T/-r 分别对应创建、查询、删除三个阶段，-i 3 表示每阶段独立重复三次，取 Mean 作为主指标、Std Dev 作为噪声判据。hard 与 easy 的区别由 -u 参数控制：默认所有 rank 在同一父目录下操作（hard），加上 -u 后各 rank 在独立子目录下操作（easy）。本节默认讨论 hard 模式数据，easy 模式数据用于对照。每次 mdtest 调用前，服务端和所有客户端同步执行 `echo 3 > /proc/sys/vm/drop_caches`，以清除上一轮遗留的页缓存。

本节涉及的编排脚本、各被测系统的切换与初始化脚本、mdtest 参数模板，以及下文各节所引数据的原始 mdtest 与 pjdfstest 输出，均按被测系统和并发档位归档于本文配套仓库的 testing/bench-v3/ 目录（仓库地址：<https://github.com/csjpgg/rucksfs>）。

5.3.4 测量边界说明

下文数据在解读前，有几点边界需要先予以说明。

空文件负载。 mdtest 的启动参数为 `-w 0`，每个文件在 `create` 之后直接 `close`，不写入任何数据。这样做是为了将元数据路径与数据路径分离，只考察元数据的表现。代价是 `DataSeter` 基本不参与测试——它仅在 `create` 时分配一个空的 `DataLocation`，在 `unlink` 时接到一次「可以回收」的通知。真实混合读写负载下 `read/write` 的表现不在本节关注范围之内。

DataSeter 最小实现。 如第 4.4 节所述，`RawDiskDataStore` 以固定偏移公式直接映射 `inode`，没有块映射，也没有空间回收，`delete_data` 目前仍是空操作。因此 `unlink` 的吞吐中有一部分来自「当前未执行真正的数据回收」这一事实，存在一定虚高。下文讨论 `remove` 时会再展开。

未提供副本容错。 `RucksFS` 目前的部署是单 `MetadataServer` 加单默认 `DataSeter`，没有副本，也没有 `Raft`。`JuiceFS+TiKV` 即使部署为单节点，`TiKV` 内部仍然运行 `Raft` 和 `Percolator` 两阶段提交^[40]，这些开销原本是为多副本和跨节点事务准备的。因此下文看到的 `RucksFS` 对 `JuiceFS+TiKV` 的性能优势，一部分源于省去了这些提交层开销，不能据此推出 `RucksFS` 作为完整分布式系统已在各方面超过 `JuiceFS+TiKV`。

在明确上述前提之后，下文数据主要用于回答两个问题：其一，`RucksFS` 在元数据路径上相对 `NFS` 与 `JuiceFS+TiKV` 的横向位置；其二，`DeltaOp` 这一具体机制在高并发下究竟发挥了多大作用。

5.3.5 横向对比： $N \in \{2, 8, 32\}$

本节把 `RucksFS-Delta` 与 `NFS`、`JuiceFS+TiKV` 放在同一套硬件、相同 `mdtest hard` 模式下运行 `create`、`stat`、`remove` 三类操作。 $N \in \{2, 8, 32\}$ 三档内三套系统都能给出可以相互比较的结果； $N \geq 64$ 时情况有变，先作交代。

$N \geq 64$ 时，`JuiceFS+TiKV` 方面，单节点 `TiKV` 在大量客户端同时挂载 `FUSE` 时会出现连接超时，生产环境本身也建议三节点起步，这里未补测。`NFS` 方面， $N = 32$ `hard` 模式下 `create` 的吞吐为 1733 ops/s，相比 $N = 2$ 的 1038 ops/s 有缓慢增长，但与同档 `easy` 模式的 6445 ops/s 相比差距很大；共享父目录下的写路径已经被 `ext4` 的 `i_rwsem` 锁限制住，这一形态在更大并发下不会改变。因此横向对比主要讨论 $N \leq 32$ 的数据， $N = 64$ 与 $N = 96$ 的 `RucksFS` 数据留到下一

节的 Delta 与 NoDelta 对照中使用。

表 5-5-5-7 分别给出 create、stat、remove 三类操作在三档横向对比下的吞吐量。每个数值为三次 iteration 的 Mean，所有单元格的 Std Dev 均在 Mean 的 5% 以内，测量稳定。下面分三段展开。

表 5-5 文件创建吞吐量 (hard 模式, 单位 ops/s)

N	RucksFS-Delta	NFS	JuiceFS+TiKV
2	542	1,038	370
8	2,126	1,752	1,401
32	8,035	1,733	4,710

表 5-6 文件查询吞吐量 (hard 模式, 单位 ops/s)

N	RucksFS-Delta	NFS	JuiceFS+TiKV
2	678	1,661	562
8	2,626	6,142	2,200
32	10,007	24,696	8,022

表 5-7 文件删除吞吐量 (hard 模式, 单位 ops/s)

N	RucksFS-Delta	NFS	JuiceFS+TiKV
2	555	1,053	308
8	2,170	3,131	1,157
32	8,245	5,682	3,728

创建吞吐。 三套系统的 create 吞吐曲线见图 5-1。 $N = 2$ 时顺序为 NFS (1 038)、RucksFS-Delta (542)、JuiceFS+TiKV (370)； $N = 8$ 时变为 RucksFS-Delta (2 126)、NFS (1 752)、JuiceFS+TiKV (1 401)； $N = 32$ 时变为 RucksFS-Delta (8 035)、JuiceFS+TiKV (4 710)、NFS (1 733)。三条曲线的相对位置随 N 发生改变，这是本段需要解读的主要现象。

对 JuiceFS+TiKV, RucksFS 在三档下的比值分别为 1.47、1.52、1.71，较为稳定。差距的来源有两条。其一，RucksFS 采用单机直连 RocksDB，一次 create 在

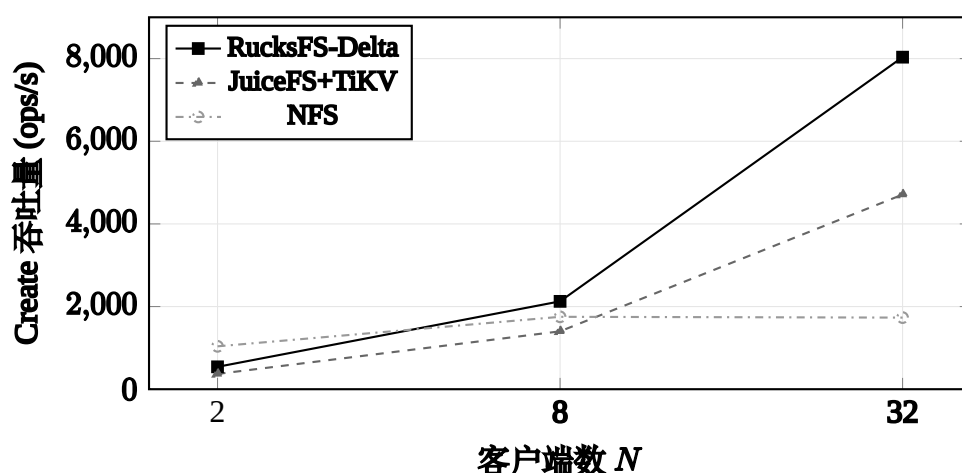


图 5-1 共享父目录场景下文件创建吞吐量随客户端数变化

MetadataServer 内即一次 AtomicWriteBatch 提交，多类键共享一次 WAL 同步即可写入；而 TiKV 需要走完整的 Percolator 两阶段提交，再经过一遍 Raft 日志，即便单节点部署、没有跨节点复制，Raft 这一层也照常执行。这部分提交层开销正是 RucksFS 所省去的。其二，DeltaOp 机制把父目录属性更新从「读-改-写」改为对独立列族的追加写，使共享父目录下的 create 事务不再在父 inode 键上竞争写锁； $N = 32$ hard 模式下 RucksFS 仍保持与 easy 模式接近的吞吐（8035 vs 8028）。JuiceFS+TiKV 没有类似机制，共享父目录更新走 MVCC 的常规读写路径， $N = 32$ hard/easy 为 4710 vs 3878，受并发影响在本档位不明显。此处 1.5–1.7 倍的差距已是较为保守的数字：JuiceFS 所用的 TiKV 为单节点部署，已省去多副本复制开销；若按生产环境推荐的三节点部署，这一比值还会更大。

对 NFS，RucksFS 的比值从 $N = 2$ 的 0.52 上升到 $N = 8$ 的 1.21，再到 $N = 32$ 的 4.64。 $N = 2$ 时 NFS 领先，原因在于一次 NFSv4.2 create 在客户端只需一次 RPC 到服务端（COMPOUND 请求内可把 LOOKUP 和 OPEN/CREATE 合并），而 RucksFS 在关闭 FUSE TTL 之后，每次 create 在客户端视角下会产生一次 lookup、一次 create、一次 release 共三次 RPC。这三次 RPC 是 VFS 语义所要求的三次事件，FUSE 框架不会合并。两侧单操作 RPC 次数之比为 1:3，与 $N = 2$ 时 1038:542 $\approx$ 1.9:1 的吞吐比大致吻合。当 N 增大时，NFS 服务端所有线程都要在父目录 inode 的 `i_rwsem` 写锁上排队，才能进入 `ext4` 进行目录项插入与 journal 提交： $N = 8$ 时 hard/easy 比为 0.48， $N = 32$ 时为 0.27，共享父目录下的 NFS create 几乎不再随 N 增长。RucksFS 的 create 在同一区间内接近线性

扩展，两者差距从 0.52 倍翻转到 4.64 倍。这一形态说明，在共享父目录并发写入场景下，主要瓶颈是父目录锁，而不是客户端与服务端之间的单次 RPC 开销；KV 方案把「目录项」和「父目录属性」拆成独立键、用不同事务路径处理，在高并发下还能继续扩展。

查询吞吐。 三档下三套系统的 stat 吞吐由高到低依次是 NFS、RucksFS-Delta、JuiceFS+TiKV。 $N = 2$ 时 NFS 的三次 iteration 分别为 1660、2076、1247，标准差达 274，波动较大，表 5-6 中 1661 为三次平均。三套系统在 stat 上的顺序与 create 不同：create 下 RucksFS 居中 ($N = 2$) 或领先 ($N = 8, 32$)，而 stat 下 NFS 始终最高。

NFS 的 stat 在三档下都稳定高于 RucksFS 约 2.3–2.5 倍。造成差距的原因有两层。第一层是路径开销。RucksFS 的一次 stat 要依次经过 FUSE 守护进程的系统调用返回、用户态 gRPC 客户端到服务端、MetadataServer 内部的 getattr 处理、RocksDB 点查，再返回折叠后的 inode 值；NFS v4.2 的 GETATTR 由内核直接处理，客户端走 kNFSD 路径，服务端在内核上下文中读 ext4 目录 htree，没有用户态切换。第二层也是更主要的一层，在存储结构本身。ext4 的目录 htree 是一棵 B+ 树的变体，点查沿哈希索引走到叶子节点，通常只需 2–3 次页访问；RocksDB 的 LSM-tree 则需要依次检查 MemTable、各层 SST 的布隆过滤器，只要某一层不确定就要读一次对应层的 SST block。即便布隆过滤器命中率接近 100%、BlockCache 也命中，一次点查涉及的代码路径仍长于 B+ 树直接走几层指针。这是 LSM 相对 B+ 树的固有代价：LSM 用更重的读路径换取更高的写吞吐。本文在第 3 章选 LSM 作为元数据引擎，正是因为面向负载中写操作出现频次远高于随机点查，目录遍历也主要走前缀扫描，LSM 的写吞吐收益大于其在读路径上付出的代价。因此 NFS 的 stat 吞吐高于 RucksFS 并不是实现不到位或配置不当，而是两种存储结构在读路径上各自擅长的体现。RucksFS 的 stat 随 N 近似线性增长 ($N = 2$ 到 $N = 32$ 约 15 倍，接近 16 倍的客户端数增长)，说明 InodeFoldedCache 与 BlockCache 两层缓存的配置在当前测试下起到了预期作用。

对 JuiceFS+TiKV，RucksFS 在 $N = 32$ hard 模式下为 1.25 倍，比 create 下的 1.71 倍有所缩小。读路径不涉及 Percolator 两阶段提交，TiKV 只需一次 BatchGet

加一次 read-index 即可返回，此时 RucksFS 单机直连 RocksDB 的优势只剩「本地调用对一次 gRPC」这一层；加上服务端维护的 InodeFoldedCache 不必每次 getattr 都重新扫描 delta_entries，仍保持了一定领先。关于 DeltaOp 在读路径上的折叠代价，下一节 Delta 与 NoDelta 对照中可以直接观察：所有档位下两种构建的 stat 差距都落在标准差之内，折叠代价在实测中观察不到。

删除吞吐。 remove 数据见表 5-7。 $N \in \{2, 8, 32\}$ 三档下 RucksFS-Delta 分别为 555、2 170、8 245 ops/s；对 JuiceFS+TiKV 的比值依次为 1.80、1.88、2.21，整体与 create 相近且随 N 略有放大；对 NFS 的比值依次为 0.53、0.69、1.45，同样呈现「低并发 NFS 领先、高并发 RucksFS 反超」的形态——单次 RPC 次数与 ext4 目录锁同样是主导因素。

在解读这些数字之前，需要说明一点：当前 RucksFS 的 DataServer 仍是最小实现，delete_data 是空操作，并不进行真正的物理空间回收。NFS 的 unlink 会触发 ext4 的 inode 位图翻转与 journal 提交，TiKV 的 unlink 需要写一条 tombstone 并最终参与后台 GC。mdtest 使用的又是空文件负载，两侧本就没有实际数据需要回收，因此这部分工作量差异在本文的测试中会被放大——remove 的倍数中有一部分是「尚未执行真正的数据回收」所带来的虚高，并非全部源于设计收益。真实的数据回收补齐后，这一倍数会相应回落。

另一方面，元数据侧的 remove 路径在 RucksFS 中已完整实现。一次 unlink 在 MetadataServer 中同样走 AtomicWriteBatch 原子提交，跨键更新按 inode 编号排序加锁，与 create 对称；父目录属性更新走 DeltaOp 追加，共享父目录下的并发 unlink 也不会在父 inode 键上互相抢锁。因此即便扣除虚高部分，元数据侧的 remove 路径仍能保持较高并发度——下一节 Delta 与 NoDelta 对照中 remove 的分叉形态也可印证这一点。

横向对比小结。 这一节的数据主要说明两件事。其一，本文基于 RocksDB 实现元数据，在相同硬件、相同客户端缓存配置下能够达到业界常见 KV 文件系统的水平——对 JuiceFS+TiKV，create/remove/stat 三项稳定领先，比值分别为 1.5–1.7、1.8–2.2 与约 1.2；差距主要来自单机直连 RocksDB 省去了 Percolator 与 Raft 两层提交开销，以及 DeltaOp 对共享父目录并发的优化。其二，相对传统本地文件系统，KV 这条路线在共享父目录高并发下更有扩展空间—— $N = 2$

时 NFS 凭借单次 RPC 路径更短、ext4 B+ 树点查更快，仍略领先 RucksFS；到 $N = 32$ 时 NFS 被 `i_rwsem` 锁限制，`create` 与 `remove` 均被 RucksFS 反超（前者约 4.6 倍，后者约 1.5 倍）；`stat` 方向 NFS 始终领先，这是 LSM 相对 B+ 树在读路径上的固有代价，本文承认这一差距，不试图在读路径上反超。横向对比只能覆盖 $N \leq 32$ ，下一节通过 Delta 与 NoDelta 的内部对照继续考察 $N \geq 64$ 的高并发区间。

5.3.6 DeltaOp 机制的内部对照

上一节横向对比主要讨论 $N \leq 32$ 的数据，原因是 $N \geq 64$ 时 JuiceFS+TiKV 和 NFS 均无法稳定给出结果。但共享父目录高并发恰好是当初设计 DeltaOp 机制所要针对的场景，因此本节专门考察 DeltaOp 在高并发下的实际作用。做法是：保留主实现不动，仅通过一个 `cargo` 特性开关关闭 DeltaOp，得到 NoDelta 构建。它与 Delta 的唯一差别在父目录属性更新这一处——NoDelta 退回传统的「读父 inode、修改、写回」方式，其余代码完全一致。两者在同一台服务器、同一批客户端、同一份负载下分别运行，直接比较。

第 3.4 节当时的设计思路是：将父目录属性更新从「读-改-写」改为向一个独立列族追加 DeltaOp，使热点父目录上的并发写入不再在同一条 inode 键上争抢写锁，从而避免高并发下的事务冲突。本节按两条线组织数据：一条沿 np 轴扫描，观察两者的吞吐曲线是否在某个请求速率处分开；另一条与 easy 模式做对照——easy 模式下每个 rank 在各自的子目录下操作，父目录键不再是写热点，若 Delta 相对 NoDelta 的差距确实来自父目录写冲突的消除，则 easy 模式下两者应当重合。

完整的并发扫描。 表 5-8 列出 Delta 与 NoDelta 在 hard 模式下沿 np 轴扫描的 `create` 吞吐，以及 easy 模式下的对照结果。

hard 模式下的数据可分两段观察。np ≤ 96 的五个档位下两者比值均为 1.00，差距在标准差范围内；np 超过 96 之后，从 128 开始 NoDelta 的吞吐降至 13 179 ops/s，后续 192、256、384 三档都稳定在 11 700–12 900 区间；Delta 则继续上升到 29 602、29 734、38 081 ops/s。两者比值依次为 1.00、1.75、2.53、2.48、2.96。两段形态与第 3.4 节的设计思路一致：低请求速率下没有父目录写冲突，

表 5-8 Delta 与 NoDelta 的 create 吞吐量扫描结果 (单位 ops/s)

N	r	np	hard (共享父目录)			easy (独立父目录)		
			Delta	NoDelta	比值	Delta	NoDelta	比值
2	1	2	542	542	1.00	539	539	1.00
8	1	8	2,126	2,122	1.00	2,120	2,117	1.00
32	1	32	8,035	8,012	1.00	8,028	7,979	1.01
64	1	64	14,726	14,773	1.00	13,645	14,696	0.93
96	1	96	20,574	20,479	1.00	20,527	20,532	1.00
64	2	128	23,042	13,179	1.75	—	—	—
96	2	192	29,602	11,703	2.53	—	—	—
64	4	256	29,734	11,985	2.48	—	—	—
96	4	384	38,081	12,855	2.96	—	—	—

是否启用 DeltaOp 结果相近；请求速率超过某一阈值之后，NoDelta 的读-改-写路径在父 inode 键上出现写冲突，Delta 不进入这一状态，两者差距被拉开。

easy 模式下两者在所有档位的比值都落在 1.00 附近。 $N = 64$ easy 档 Delta 的一次 iteration 恰落在抖动较大的区间，是个别测量噪声，不影响主体判断。easy 模式让每个 rank 在各自子目录下操作，父目录键不再是写热点——如果 Delta 相对 NoDelta 的优势来自其他原因（例如更好的内存布局或更高的 CPU 利用率等），那么 easy 模式下也应当观察到差距，而实测没有。这组对照说明 hard 模式下的差距确实来自父目录写冲突，而非测试噪声或其他无关因素。

图 5-2 将 hard 模式数据按 np 绘制为曲线，分叉形态更为直观。图中 $np \leq 96$ 的部分是 $r = 1$ 的扩展性曲线， $np > 96$ 的部分来自提高 r 值所带来的请求速率增量。

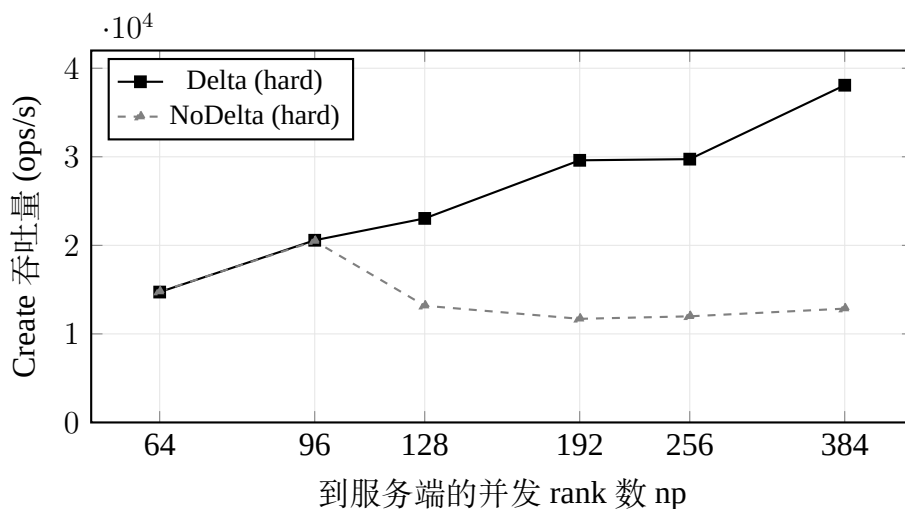


图 5-2 Delta 与 NoDelta 的 create 吞吐量对比 (hard 模式，按 np 扫描)

NoDelta 下降的原因。 NoDelta 在 np 从 96 到 128 出现骤降、之后稳定在 12 000 ops/s 附近的现象，可以从 MetadataServer 端的事务执行过程中看出。NoDelta 下，一次 create 事务内部需要完成的工作是：对父目录 inode 键调用一次 get_for_update 获取写锁，读取当前父 inode 的值，修改 mtime/ctime，再 put 回去，连同新建的目录项和 inode 一并提交。共享父目录下，所有客户端的 create 事务都在争抢同一条父 inode 键的写锁。如第 3 章所述，本文采用悲观并发控制：未取得锁的事务会等待，超时后回滚，由服务端再次重试。并发度上升后，这个「等待—回滚—重试」的循环会自我放大——被拒绝的事务需要先释放已取得的其他锁、回滚已写入的 batch，之后才能重新加入锁竞争；此时队列中又堆积了更多重试请求。请求速率超过某个阈值后，吞吐会急剧下降，之后稳定在重试循环的稳态上限。

启用 DeltaOp 后路径变化明显。create 不再改动父 inode 的值，父目录的属性变化被编码为一条 DeltaOp，写入 delta_entries 列族。DeltaOp 的键中带有单调递增的序列号 seq，每次写入的目标键互不相同，不会与其他事务在同一个键上争锁。父 inode 的值只在后台的 DeltaCompactionWorker 折叠增量时才被写一次，该动作不出现在测试的主要路径上，不影响 create 本身。锁竞争消除的同时，热点目录上的随机写也转为顺序追加。

把上述原因对应到数据上：np ≤ 96 时锁等待还未到引发重试堆积的程度，NoDelta 与 Delta 吞吐基本一致；np 超过 96 之后 NoDelta 吞吐降至 13 000 ops/s 左右，对应重试队列开始累积；继续提高请求速率（192、256、384）时 NoDelta 仍稳定在这一区间，说明系统已进入重试循环的稳态——继续加压不能让更多事务提交，反而让单次事务的实际耗时随重试次数增加。Delta 不进入这一循环，因此吞吐能够一路扩展到 38 000 ops/s，并仍有上升趋势。

这一形态还解释了「客户端数 N 」与「DeltaOp 生效阈值」之间为什么看不到直接关系。对比 $N = 64, r = 2$ (np = 128) 与 $N = 96, r = 1$ (np = 96) 两组：前者客户端少，但 NoDelta 明显下降；后者客户端多，两者反而基本持平。真正决定事务冲突是否发生的，是服务端单位时间接收到的 create 请求数，不是发起请求的客户端数量。本节沿 N 与 r 两个维度扫描的价值正在这里——把到达服务端的请求速率作为单独一个变量来控制，使得 DeltaOp 的作用可以直接对

应到这一变量上。

对 remove 和 stat 的影响。 DeltaOp 的设计目标虽然是父目录属性更新，但其影响并不限于 create。表 5-9 给出 remove 与 stat 沿 np 轴扫描下两种构建的对比。

表 5-9 DeltaOp 对 remove 与 stat 的影响 (hard 模式, 单位 ops/s)

<i>N</i>	<i>r</i>	np	remove			stat		
			Delta	NoDelta	比值	Delta	NoDelta	比值
64	1	64	14,239	15,208	0.94	18,413	18,505	1.00
96	1	96	21,673	21,311	1.02	25,743	25,869	0.99
64	2	128	24,996	13,507	1.85	29,624	29,823	0.99
96	2	192	31,836	11,374	2.80	37,718	35,820	1.05
64	4	256	27,847	12,521	2.22	39,719	39,551	1.00
96	4	384	32,882	12,500	2.63	51,613	49,746	1.04

remove 的形态与 create 相似。unlink 同样需要更新父目录的 mtime/ctime，走相同的事务流程。np ≤ 96 时 NoDelta 尚未出现下降，超过 96 之后立刻分开，稳态上限同样在 12 000 ops/s 左右。np = 192 时 remove 比值达到 2.80，稍高于同档 create 的 2.53，原因在于 unlink 事务在 NoDelta 下除了持父 inode 写锁，还需要对目标 inode 调用 get_for_update 以更新 nlink，持锁时间略长，重试堆积也相应更严重。

stat 在两种构建下吞吐一致，所有档位下比值都在 0.99–1.05 之间，差距小于标准差。这一结果需要说明 DeltaOp 在读路径上的折叠代价问题：启用 DeltaOp 之后，一次 getattr 在逻辑上除了读基值，还要扫描父目录 inode 对应的 delta_entries 并把未折叠的增量叠加回去，理论上会比直接读一个 inode 值慢。但数据显示这一代价在实测中观察不到。原因有两点：一是 InodeFoldedCache 把已折叠结果缓存下来，命中时一次内存查表即可返回，不触及 delta_entries；二是 DeltaCompactionWorker 把单个 inode 的增量条数控制在 32 以内，即便缓存未命中，叠加所需的扫描量也很小。换言之，DeltaOp 给写路径带来的收益并

未在读路径上付出可观察到的代价。

回到第 3 章的设计。将上述结果与第 3 章关于 DeltaOp 的几条说法对照，整体一致：

- 「DeltaOp 的收益在低并发下并不明显」（第 3.4 节）。np ≤ 96 时 Delta 与 NoDelta 在 create、remove、stat 三项上的比值均在 0.99–1.02 之间。DeltaOp 针对的是热点父目录上的写争抢，没有形成争抢时，走读-改-写还是走增量追加结果相近。
- 「DeltaOp 在高并发下消除父目录事务冲突」（第 3.4 节）。np 从 96 到 128 时 NoDelta 出现明显的吞吐下降，之后在 12 000 ops/s 附近形成稳态；Delta 则继续扩展至 38 000 ops/s。两条曲线的分叉点正好对应 PCC 锁等待开始累积的位置。
- 「DeltaOp 的优势只在共享父目录场景下出现」。easy 模式下两者吞吐基本一致，仅 hard 模式下存在差距。这也说明 DeltaOp 的收益并非来自其他无关因素。
- 折叠代价在读路径上观察不到。stat 在两种构建下吞吐几乎相同。这一点依赖 InodeFoldedCache 与 32 条增量上限两项配套设计，二者共同使得折叠动作的实际成本在当前测量下无法体现出来。

上述四点合起来说明，第 3 章所述的 DeltaOp 机制在分布式部署下确实发挥了作用——差异出现的方向与幅度、与请求速率的关系、以及 easy/hard 两种场景的对比，都与当初的设计吻合。

5.4 本章小结

本章沿两条线检验了前文的设计。

POSIX 合规性方面，将 pjdftest 的 398 条用例按本文实现范围分解：扣除 mknod/mkfifo 相关用例（167 条）、DataServer 单文件大小用例（1 条）和 pjdftest 框架自身跳过的用例（48 条）之后，剩下 182 条「实现范围内」用例全部通过，通过率 100%。打开后 unlink、硬链接覆盖、长名称错误码、O_TRUNC 时间戳、事务内空目录检查等边界用例同样在通过列表中，正好对应第 3 章的跨列族原子提交、句柄生命周期维护、统一加锁顺序以及客户端协调路径。mknod/mkfifo 的

不实现是面向典型分布式存储负载所作的显式取舍，接口统一返回 EOPNOTSUPP，不以错误来遮盖真实问题。

性能方面，横向对比主体数据来自 $N \in \{2, 8, 32\}$ 三档下对 NFS 与 JuiceFS+TiKV 的比较。对 JuiceFS+TiKV，RucksFS 在 create/remove/stat 三项上稳定领先——create 1.5–1.7 倍、remove 1.8–2.2 倍、stat 约 1.2 倍；主要差距来自单机直连 RocksDB 省去了 Percolator 与 Raft 两层提交开销，以及 DeltaOp 对共享父目录并发的优化。对 NFS， $N = 2$ 时 NFS 凭借单次 RPC 路径更短与 ext4 B+ 树点查更快，三项均略领先 RucksFS；随 N 增大，i_rwsem 限制共享父目录的写扩展， $N = 32$ 时 create 被 RucksFS 反超约 4.6 倍、remove 约 1.5 倍；stat 方向 NFS 始终领先，这是 LSM 相对 B+ 树在读路径上的固有代价，本文承认这一差距。横向对比只能覆盖 $N \leq 32$ ： $N \geq 64$ 时 NFS 与 JuiceFS+TiKV 分别受 ext4 目录锁和单节点 TiKV 连接规模的限制，继续比较意义有限。

高并发区间的结果由 RucksFS 自身的 Delta 与 NoDelta 对照给出。两种构建仅差一个编译开关，其余完全一致，因此二者差距只能来自父目录更新路径的不同。沿 np 轴扫描六档：np ≤ 96 时两者持平，接近线性扩展；np 超过 96 之后 NoDelta 进入「等待—回滚—重试」循环，稳态吞吐停在 12 000 ops/s 附近，Delta 则继续扩展到 38 000 ops/s 并仍有上升趋势。create/remove 方向上两者比值依次为 1.00、1.75、2.53、2.48、2.96 与 0.94、1.02、1.85、2.80、2.22、2.63；stat 方向上两者在所有档位都基本持平（0.99–1.05），说明 DeltaOp 的折叠代价在读路径上观察不到——InodeFoldedCache 与 32 条增量上限两项配套设计使得折叠动作的实际成本在当前测量下无法体现出来。easy 模式下两者完全一致。这组对比说明：DeltaOp 在共享父目录高请求速率场景下确实消除了父 inode 的写冲突，这正是第 3 章当时要解决的问题。

6 总结与展望

6.1 工作总结

本文围绕“基于 KV 存储管理文件系统元数据”这一具体问题，设计并实现了 RucksFS——一个以 RocksDB 为元数据引擎、通过 FUSE 对外提供 POSIX 接口的分布式文件系统原型，并完成了较为完整的实验评估。具体完成的工作如下：

- (1) 实现了基于 RocksDB 多列族的 FUSE 文件系统 RucksFS，支持 `create`、`mkdir`、`unlink`、`rmdir`、`rename`、`readdir`、`setattr`、`read`、`write` 等常用 POSIX 操作。元数据按访问模式划分到 `inodes`、`dir_entries`、`delta_entries`、`system` 四个列族；目录项键采用「父 inode + 子文件名」的大端序拼接编码，使文件操作基本对应目录树上一条边的增删。
- (2) 引入了增量更新与懒折叠机制 (DeltaOp)。父目录属性的修改不再就地重写基值，而是追加一条 DeltaOp，读路径在读取时再将基值和增量合成；后台线程在增量条数超过 32 时执行折叠。该机制借鉴了 Mantle 的 Delta Record 思路，但实现上更为轻量，整个流程都在单个 RocksDB 实例内完成。
- (3) 基于 RocksDB 悲观事务 (PCC) 处理元数据并发。所有元数据变更操作通过事务完成加锁与冲突检测，以原子提交保证多键更新的一致性；跨 inode 事务按编号排序加锁以避免死锁；锁等待超时后服务端自动重试最多 3 次（起始退避 50 μ s 并逐次翻倍），失败后才以 EAGAIN 向上层返回。
- (4) 客户端基于 fuse3 0.8（启用 `tokio-runtime` 与 `unprivileged` 特性）与 `tokio` 异步运行时实现，挂载时启用 `default_permissions` 与 `allow_other`，常规权限检查交由内核 VFS 完成。
- (5) 在 1 台 64 核服务器加最多 96 台 2 核客户端的腾讯云集群上完成了系统评估。POSIX 合规性方面，`pjdfstest` 套件共 398 条用例，按本文实现范围分解后剩下 182 条「实现范围内」用例在分布式部署下全部通过，通过率 100%。性能方面分两段： $N \in \{2, 8, 32\}$ 三档下进行三方对比，RucksFS 相对 JuiceFS+TiKV 在 `create/remove/stat` 三项上分别领先约 1.5–1.7 倍、1.8–

2.2 倍与 1.2 倍；相对 NFS 在 $N = 2$ 处因关闭客户端缓存、单次 create 多走两次 RPC 而略低于 NFS，但到 $N = 32$ 时 ext4 的 `i_rwsem` 锁开始主导，RucksFS 在 create 上反超 NFS 约 4.6 倍；stat 方向 NFS 始终领先 RucksFS，原因是 ext4 B+ 树点查与内核 NFS 路径在读取上更为直接。 $N \geq 64$ 时 NFS 和 JuiceFS+TiKV 均无法给出稳定的数据，此时转用 Delta 与 NoDelta 两个构建变体互相对照，沿 $np = N \times r$ 两个维度扫描——np 超过 96 后 NoDelta 进入事务冲突循环、稳态吞吐保持在 12 000 ops/s 附近，Delta 则继续扩展至 38 000 ops/s， $np = 384$ 时 create 比值 2.96、remove 比值 2.63；easy 模式下两者基本一致。

6.2 不足与展望

前述数据说明 RucksFS 在元数据关键路径上已经达到了当初设定的目标，但作为毕业设计阶段的原型，系统仍有几项明显不足。下列几项按优先级由高到低列出。

- (1) **DataServer 仍是最小实现。**当前 RawDiskDataStore 使用固定偏移公式直接映射 inode，`delete_data` 是空操作，不做空间回收，单文件存在 64 MiB 的逻辑上限。实验中使用 `mdtest` 的空文件负载可以跑通，但要作为真实可用的文件系统，还需要补上块分配、空间回收以及对大文件的支持。
- (2) **客户端缓存策略偏保守。**本文为了对元数据路径做纯粹测量，把 FUSE 的 `entry/attr TTL` 设为 0，使每次 `lookup` 和 `getattr` 都穿透到服务端。这一选择的代价在第 5.3.5 节已有体现： $N = 2$ 时 RucksFS 对 NFS 的 create 比值只有 0.52，主要原因就是每次 create 多走了一次 `lookup` 和一次 `release`，而其中 `lookup` 的结果在大多数应用访问模式下是稳定的，本可缓存。后续可以从三个方向展开：其一，把 TTL 放宽到秒级以吸收重复的 `getattr`；其二，为「名字不存在」的结果维护客户端负缓存，避免 `create-before-check` 这类语义反复触达服务端；其三，补一个目录失效协议——客户端缓存在目录发生 `rename`、`unlink` 时能够收到服务端主动推送的失效通知，而不是依赖 TTL 被动超时。前两项可通过调参实现，第三项需要协议层面的设计，工程量相对较大，但在共享目录场景下价值明显。

(3) **跨服务 RPC 合并尚不充分**。一次 `create` 目前产生三次跨节点 RPC：一次 `lookup` 确认名字不冲突、一次 `create` 执行元数据事务、一次 `release` 释放句柄。NFS v4 通过 COMPOUND 请求把 LOOKUP 与 CREATE 打包到一次往返中，这也是低并发下 NFS 领先 RucksFS 的主要原因。本文已将 `create` 与 `open` 合并为 `create_and_open` 一次 RPC，下一步还可以从三个方向继续推进：一是把 `lookup` 合入 `create`，客户端在 `open(O_CREAT)` 语义下本就允许「先检查后创建」在一次请求中完成；二是引入批量 `create` 接口，一次 RPC 创建 K 个文件，均摊网络开销，对编译产物、日志归档等大批量写入场景意义直接；三是把 `readdirplus` 之后常见的批量 `setattr`、`unlink` 同样做成批量接口。这些改动都不触及元数据存储模型本身，只在客户端与服务端之间的协议层面做工程重构，收益相对确定。

致 谢

落笔至此，四年的本科时光已近尾声。从最初面对一片空白的 `main.rs` 不知从何下手，到最终在真实分布式集群上把数字一行行跑出来，回望这段路，要感谢的人很多。

最先想到的自然是我的指导老师。初次与老师讨论课题时，我对 KV 存储与文件系统元数据之间的关系只有最朦胧的想法；是老师一次次把我从过于宏大的叙事拉回到具体的问题上——究竟要写哪条键？究竟该在事务里解决什么？究竟是 `DeltaOp` 起了作用，还是缓存起了作用？论文中的每一处改动、每一张表格、每一次结论的审慎措辞，背后都有老师的耐心批注与反复推敲。陆游有句诗，写系统工作再贴切不过——“纸上得来终觉浅，绝知此事要躬行”。本文若说有一点真切的体会，便是这一句。

也要感谢实验室的几位学长学姐。在我初次把代码跑出 `core dump` 时、在 `RocksDB` 事务冲突反复出现时、在远程测试脚本因为 `SSH` 配置问题一次又一次超时，他们都曾放下自己手头的工作来帮我看日志、讲原理、甚至直接替我改配置。许多细节在论文里已经化作了简短的一句说明，但这些时刻是真实存在过的，我也会记住。和他们的每一次讨论都让我意识到，科研不是一个人的事，一个好问题背后总是站着许多愿意一起思考的人。

感谢学校和学院四年来的培养。图书馆里安静的桌椅、实验课上第一次自己编译出的内核、深夜空无一人的机房、以及同学之间那些关于“这玩意到底能不能跑起来”的反复争论，共同构成了我工程素养的底色。大学并不只是知识的集合，更是一段把自己从“会做题的学生”慢慢变成“能解决问题的工程师”的过程。这一过程不会在毕业典礼那天戛然而止，但大学给了它最初的方向。

最后要深深感谢我的父母。写论文这段日子里，常常是通宵调代码到凌晨，白天又需要补睡再起来继续实验。父母从未催促过我什么，只是在每次视频时默默问一句“最近累不累”，在我难得回家的几天里准备好我爱吃的菜。他们或许并不完全理解 `inode`、`LSM-tree`、`PCC` 事务这些我口中的名词，但他们理解他们的孩子正在做一件自己认真对待的事情，这就足够了。他们的支持是我四年求学路上最稳的底，也是我接下来走入社会的底气。

走到今天，仍有许多未知在前方。但我也相信，只要还愿意在每一个具体的问题前坐下来、把一行字敲下去、把一次测试跑通，路就会一点一点延伸出来。谨以这篇论文，致我所遇见的每一位师长、同伴与家人。

参考文献

- [1] REN K, GIBSON G. TABLEFS: Enhancing Metadata Efficiency in the Local File System[C/OL]// 2013 USENIX Annual Technical Conference (USENIX ATC 13). San Jose, CA: USENIX Association, 2013: 145–156. <https://www.usenix.org/conference/atc13/technical-sessions/presentation/ren>.
- [2] LI J, CAO B, JIAN J, et al. Mantle: Efficient Hierarchical Metadata Management for Cloud Object Storage Services[C]// Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP '25). [S.l.]: ACM, 2025: 928–943. doi: 10.1145/3731569.3764824.
- [3] O'NEIL P, CHENG E, GAWLICK D, et al. The Log-Structured Merge-Tree (LSM-Tree)[J]. Acta Informatica, 1996, 33(4): 351–385. doi: 10.1007/s002360050048.
- [4] DEAN J, GHEMAWAT S. LevelDB: A Fast Key-Value Storage Library. <https://github.com/google/leveldb>. Accessed: 2026. 2011.
- [5] DONG S, KRYCZKA A, JIN Y, et al. Evolution of Development Priorities in Key-Value Stores Serving Large-Scale Applications: The RocksDB Experience[C/OL]// 19th USENIX Conference on File and Storage Technologies (FAST 21). [S.l.]: USENIX Association, 2021: 33–49. <https://www.usenix.org/conference/fast21/presentation/dong>.
- [6] Apache Software Foundation. Apache Ozone: A Scalable, Redundant, Distributed Object Store for Hadoop. <https://ozone.apache.org>. Accessed: 2026. 2021.
- [7] PAN S, STAVRINOS T, ZHANG Y, et al. Facebook's Tectonic Filesystem: Efficiency from Exascale[C/OL]// 19th USENIX Conference on File and Storage Technologies (FAST 21). [S.l.]: USENIX Association, 2021: 1–16. <https://www.usenix.org/conference/fast21/presentation/pan>.

- [8] IEEE, The Open Group. IEEE Std 1003.1-2017 — POSIX.1-2017: Standard for Information Technology — Portable Operating System Interface (POSIX)[R/OL]. Standard. IEEE, 2017. <https://pubs.opengroup.org/onlinepubs/9699919799/>.
- [9] Juicedata, Inc. JuiceFS: A POSIX-Compatible Shared Filesystem for Cloud. <https://juicefs.com>. Open-source, available at <https://github.com/juicedata/juicefs>. 2021.
- [10] LIU H, WANG Q, YANG Y, et al. CFS: A Distributed File System for Large Scale Container Platforms[C]// Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19). [S.l.]: ACM, 2019. doi: 10.1145/3299869.3314046.
- [11] KLABNIK S, NICHOLS C. The Rust Programming Language. <https://doc.rust-lang.org/book/>. Also published by No Starch Press, ISBN 978-1-7185-0310-6. 2023.
- [12] REN K, ZHENG Q, PATIL S, et al. IndexFS: Scaling File System Metadata Performance with Stateless Caching and Bulk Insertion[C]// Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '14). Best Paper Award. [S.l.]: IEEE Press, 2014: 237–248. doi: 10.1109/SC.2014.25.
- [13] LI S, LU Y, SHU J, et al. LocoFS: A Loosely-Coupled Metadata Service for Distributed File Systems[C]// Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '17). [S.l.]: ACM, 2017: 1–12. doi: 10.1145/3126908.3126928.
- [14] XU Q, ARUMUGAM R V, YONG K L, et al. Efficient and Scalable Metadata Management in EB-Scale File Systems[J]. IEEE Transactions on Parallel and Distributed Systems, 2014, 25(11): 2840–2850. doi: 10.1109/TPDS.2013.293.
- [15] GHEMAWAT S, GOBIOFF H, LEUNG S.-T. The Google File System[C]// Proceedings of the 19th ACM Symposium on Operating Systems Principles (SOSP '03). [S.l.]: ACM, 2003: 29–43. doi: 10.1145/945445.945450.

- [16] SHVACHKO K, KUANG H, RADIA S, et al. The Hadoop Distributed File System[C]// 2010 IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST). [S.l.]: IEEE, 2010: 1–10. doi: 10.1109/MSST.2010.5496972.
- [17] WEIL S A, BRANDT S A, MILLER E L, et al. Ceph: A Scalable, High-Performance Distributed File System[C/OL]// Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06). [S.l.]: USENIX Association, 2006: 307–320. <https://www.usenix.org/conference/osdi-06/ceph-scalable-high-performance-distributed-file-system>.
- [18] DECANDIA G, HASTORUN D, JAMPANI M, et al. Dynamo: Amazon's Highly Available Key-Value Store[C]// Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP '07). [S.l.]: ACM, 2007: 205–220. doi: 10.1145/1294261.1294281.
- [19] GUO J, GUO L, SHI T, et al. SingularFS: A Billion-Scale Distributed File System Using a Single Metadata Server[C]// Proceedings of the 2023 USENIX Annual Technical Conference (ATC '23). [S.l.]: USENIX Association, 2023.
- [20] WANG Y, WU Y, LI C, et al. CFS: Scaling Metadata Service for Distributed File System via Pruned Scope of Critical Sections[C]// Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23). [S.l.]: ACM, 2023: 331–346. doi: 10.1145/3552326.3587443.
- [21] YANG Y, CAO Q, YANG L, et al. Batch-File Operations to Optimize Massive Files Accessing: Analysis, Design, and Application[J]. ACM Transactions on Storage, 2020, 16(3): 1–30. doi: 10.1145/3404119.
- [22] CAO X, PATEL S, LIM S.-Y, et al. FetchBPF: Customizable Prefetching Policies in Linux with eBPF[C]// Proceedings of the 2024 USENIX Annual Technical Conference (ATC '24). [S.l.]: USENIX Association, 2024.

- [23] TABATABAI B, III J C S, SWIFT M M. FBMM: Making Memory Management Extensible With Filesystems[C]// Proceedings of the 2024 USENIX Annual Technical Conference (ATC '24). [S.l.]: USENIX Association, 2024: 901–917.
- [24] HEIDARI A, AHMADI A, ZHI Z, et al. MetaHive: A Cache-Optimized Metadata Management for Heterogeneous Key-Value Stores. 2024. arXiv: 2407.19090 [cs.DB]. <https://arxiv.org/abs/2407.19090>.
- [25] SHEN J, ZUO P, LUO X, et al. FUSEE: A Fully Memory-Disaggregated Key-Value Store[C/OL]// 21st USENIX Conference on File and Storage Technologies (FAST 23). Santa Clara, CA: USENIX Association, 2023: 81–98. <https://www.usenix.org/conference/fast23/presentation/shen>.
- [26] DONG C, WANG F, YANG Y, et al. Low-Latency and Scalable Full-path Indexing Metadata Service for Distributed File Systems[C]// Proceedings of the 41st IEEE International Conference on Computer Design (ICCD 2023). [S.l.]: IEEE, 2023. doi: 10.1109/ICCD58817.2023.00051.
- [27] XU J, DONG M, TIAN Q, et al. SwitchFS: Asynchronous Metadata Updates for Distributed Filesystems with In-Network Coordination[C]// Proceedings of the 21st European Conference on Computer Systems (EuroSys '26). [S.l.]: ACM, 2026. doi: 10.1145/3767295.3769349.
- [28] 龚拓宇. 基于 RDMA 的集群文件系统元数据高速存取技术[D]. 清华大学, 2020.
- [29] 范立衡, 任祖杰. 基于键值存储的元数据集群副本一致性研究[J]. 杭州电子科技大学学报, 2014, 34(2): 89–93.
- [30] 吴雨飞, 李诚, 李嘉豪, 等. DFS 元数据缓存一致性的轻量级维护机制[J]. 计算机系统应用, 2025, 34(10): 1–8.
- [31] SZEREDI M. FUSE: Filesystem in Userspace. <https://github.com/libfuse/libfuse>. Linux kernel module since version 2.6.14. 2005.
- [32] HOLO S. Fuse3: Async FUSE library for Rust built on tokio. <https://crates.io/crates/fuse3>. Rust crate for FUSE, version 0.8. 2024.

- [33] VANGOOR B K R, TARASOV V, ZADOK E. To FUSE or Not to FUSE: Performance of User-Space File Systems[C/OL]// 15th USENIX Conference on File and Storage Technologies (FAST 17). Santa Clara, CA: USENIX Association, 2017:59–72. <https://www.usenix.org/conference/fast17/technical-sessions/presentation/vangoor>.
- [34] ROSENBLUM M, OUSTERHOUT J K. The Design and Implementation of a Log-Structured File System[C]// Proceedings of the 13th ACM Symposium on Operating Systems Principles (SOSP '91). Also published in ACM Transactions on Computer Systems, 10(1):26–52, 1992. [S.l.]: ACM, 1992:1–15. doi: 10 . 1145/121132.121137.
- [35] CHANG F, DEAN J, GHEMAWAT S, et al. Bigtable: A Distributed Storage System for Structured Data[C/OL]// Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI '06). [S.l.]: USENIX Association, 2006:205–218. <https://www.usenix.org/conference/osdi-06/bigtable-distributed-storage-system-structured-data>.
- [36] LU L, PILLAI T S, ARPACI-DUSSEAU A C, et al. WiscKey: Separating Keys from Values in SSD-Conscious Storage[C/OL]// 14th USENIX Conference on File and Storage Technologies (FAST 16). [S.l.]: USENIX Association, 2016: 133–148. <https://www.usenix.org/conference/fast16/technical-sessions/presentation/lu>.
- [37] Meta Platforms, Inc. RocksDB Wiki. <https://github.com/facebook/rocksdb/wiki>. Accessed: 2026. 2023.
- [38] DAWIDEK P J. Pjdfstest: POSIX File System Test Suite. Comprehensive POSIX compliance test suite for file systems. 2013. <https://github.com/pjd/pjdfstest>.
- [39] IOR/mdtest Development Team. Mdtest: HPC Metadata Benchmark. Part of the IOR project, used by IO500 for storage system ranking. 2024. <https://github.com/hpc/ior>.

- [40] PENG D, DABEK F. Large-Scale Incremental Processing Using Distributed Transactions and Notifications[C/OL]// Proceedings of the 9th USENIX Symposium on Operating Systems Design and Implementation (OSDI '10). [S.l.]: USENIX Association, 2010: 1–15. https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Peng.pdf.